PROJECT REPORT

CyberTrace

Digital Forensics Timeline Analyzer

Log ingestion, evidence preservation, UTC-normalized timeline reconstruction, rule-based correlation and endpoint-agent collection

Field	Detail
Project title	CyberTrace: Digital Forensics Timeline Analyzer
Team	Team Ignite (team code A3-PS013-TC189)
Team members	Kishore Parasuraman, Jabez Gabrilraj
Problem statement	PS-013, Incident Timeline and Evidence Collection Support
Institution	Karunya Institute of Technology and Sciences
Academic year	2026-27
Version reported	0.4.3

Abstract

Investigators who work from logs usually have to deal with files scattered across hosts, timestamps in different formats and zones, and no record of how the evidence was handled. CyberTrace is a digital forensics timeline analyzer built for small investigation teams to address this.

The system accepts Linux authentication logs and Apache or nginx access logs, either by upload or through an optional Linux endpoint agent. It preserves each original file unchanged with a SHA-256 hash and a manifest, parses the records into a unified timeline, and converts timestamps to UTC only when the source or the investigator supplies enough information. Anything else is flagged, not guessed. Related events from different sources are linked by transparent rules that report a confidence level and an explanation. A quality dashboard shows collection coverage and gaps, parsing failures, timestamp uncertainty, duplicates, integrity status and an evidence audit trail. Findings can be exported as a PDF report and CSV datasets.

CyberTrace is implemented with Python, FastAPI, PostgreSQL, SQLAlchemy and Alembic, with a dependency-free browser interface and Docker deployment files. This report covers version 0.4.3: the requirements, the architecture, the implementation, and the results of 81 backend tests, 10 agent tests, a 31-check synthetic end-to-end run and a demonstration investigation of 8 evidence files, 29 events and 52 correlations. The system is a prototype for local or trusted-network use and is not a production forensic platform.

Keywords: digital forensics, timeline analysis, log correlation, evidence integrity, SHA-256, timestamp normalization, endpoint agent

Table of Contents

Chapter 1. Introduction **5**
1.1 Background 5
1.2 Problem Statement 5
1.3 Motivation 5
1.4 Objectives 6
1.5 Scope 6
1.6 Development Phases 6
1.7 Report Organization 6
Chapter 2. Literature Review and Positioning **8**
2.1 Digital Forensics Timeline Analysis 8
2.2 Log Parsing and Normalization 8
2.3 Evidence Integrity and Chain of Custody 8
2.4 Related Tools 8
2.5 Gap Analysis 9
2.6 Positioning 9
Chapter 3. System Analysis **10**
3.1 Existing Process and Its Limitations 10
3.2 Proposed System 10
3.3 Functional Requirements 10
3.4 Non-Functional Requirements 11
3.5 Use Cases 11
3.6 Investigation Workflow 11
3.7 Constraints and Assumptions 11
Chapter 4. System Design **13**
4.1 Architecture Overview 13
4.2 Deployment Architecture 13
4.3 Backend Component Architecture 14
4.4 Evidence Processing Data Flow 15
4.5 Data Model 16
4.6 Timestamp Handling 17
4.7 Correlation Heuristic 18
4.8 Endpoint Agent Protocol 19
4.9 Evidence Preservation and Integrity 19
4.10 Collection Coverage and Gaps 20
4.11 Security Design 20
Chapter 5. Implementation **21**
5.1 Technology Stack 21
5.2 Repository Layout 21
5.3 Backend 21
5.4 Parsers and Timestamp Resolution 22
5.5 Evidence Storage and Integrity 22
5.6 Frontend 22
5.7 Endpoint Agent 23
5.8 Reporting and Export 23
5.9 Deployment 23
5.10 Key Code Excerpts 23
Chapter 6. Testing and Evaluation **24**
6.1 Test Strategy 24
6.2 Backend Tests 24

6.3 Agent Tests 25
6.4 Synthetic End-to-End Check 25
6.5 Synthetic Scenarios 25
6.6 Quality Metrics on the Demonstration Dataset 26
6.7 Docker Verification 26
6.8 Performance Notes 26
6.9 Interpretation of Results 26
Chapter 7. Results and Discussion 27
7.1 Dashboard 27
7.2 Sources, Gaps and Integrity 27
7.3 Correlations 29
7.4 Timeline and Event Detail 29
7.5 Quality Page 31
7.6 Agent Fleet 38
7.7 Workspace 38
7.8 Discussion 40
Chapter 8. Conclusion, Limitations and Future Work 41
8.1 Conclusion 41
8.2 Limitations 41
8.3 Future Work 41
8.4 Team 42
References 43
Appendix A. API Endpoint Inventory 44
Appendix B. Database Tables 46
Appendix C. Test Inventory 47
Appendix D. Deployment Notes 48
D.1 Server 48
D.2 Agent 48
Appendix E. Agent Configuration Reference 49

List of Figures

Figure 4.1 System architecture: layers and dependencies 13
Figure 4.2 Deployment architecture and trust boundaries 14
Figure 4.3 Backend component architecture 15
Figure 4.4 Evidence processing data flow 16
Figure 4.5 Core data model (simplified) 16
Figure 4.6 Correlation engine workflow 18
Figure 4.7 Endpoint agent collection protocol 19
Figure 7.1 Dashboard summary metrics 27
Figure 7.2 Dashboard overview panel 27
Figure 7.3 Evidence sources and collection gaps 28
Figure 7.4 Coverage gaps 28
Figure 7.5 Integrity and hash verification 28
Figure 7.6 Correlations panel 29
Figure 7.7 Unified timeline 30
Figure 7.8 Event detail drawer 31
Figure 7.9 Quality summary metrics 31
Figure 7.10 Sources and collection gaps 32
Figure 7.11 UTC normalization 33
Figure 7.12 Preservation checks 34
Figure 7.13 Evidence lifecycle audit (shown in three parts) 37
Figure 7.14 Agent list 38
Figure 7.15 Agent detail: batches, sources and errors 38
Figure 7.16 Workspace heading 38
Figure 7.17 Workspace metrics 39
Figure 7.18 Import evidence form 39
Figure 7.19 Collected files 39
Figure 7.20 Timeline panel 40

Chapter 1. Introduction

1.1 Background

Most incident investigations do not fail for lack of data. They fail because the data is scattered. An authentication log sits on one server, a web access log on another, and each uses its own timestamp convention, its own way of naming a host and its own idea of what a user is. Before anyone can say what happened and in what order, the evidence has to be gathered, protected from accidental change, brought onto one time scale and read side by side.

Small investigation teams, such as a college security club, a lab or a small IT department, usually do this by hand. They copy files, open them in a text editor, build a spreadsheet and write the report afterwards. The work is slow, and the result is hard to defend because nothing records how the evidence was handled along the way.

CyberTrace is a digital forensics timeline analyzer built for this situation. It takes log evidence from manual upload or from a small Linux agent, preserves the original bytes with a SHA-256 hash, parses the records into a single timeline, and links related events across sources using rules that can be read and explained. It also reports what it does not know: missing collection periods, timestamps that could not be resolved, records that failed to parse and files whose hash no longer matches.

The project was developed by Team Ignite for problem statement PS-013 (Incident Timeline and Evidence Collection Support) of the AIORI-3 Hackathon. This report describes version 0.4.3 of the system.

1.2 Problem Statement

Given a set of logs from one or more hosts, an investigator needs to do four things:

1. preserve the original files and be able to show later that they have not been altered;
2. build one chronological view in which every timestamp is either defensible or clearly marked as uncertain;
3. find candidate relationships between events that come from different sources; and
4. state honestly where the evidence is incomplete.

Ad-hoc scripts and spreadsheets cover parts of this. None of the commonly used ones combine preservation, normalization, correlation and quality reporting in a single tool that also keeps an audit trail.

1.3 Motivation

The following difficulties came up repeatedly when the team looked at how small teams handle logs during an incident.

- Log files are spread across several machines and have to be collected by hand.
- Timestamps differ in format and time zone. Classic syslog lines carry no year and no zone at all.
- Malformed or unusual lines are easy to skip without noticing.
- A gap in the collected logs can look the same as a quiet period on the host.
- It is hard to show afterwards that an evidence file was not edited.
- Links between, for example, failed SSH logins and later web requests from the same address are found by searching manually.
- Findings are recorded in different ways from one case to the next.

1.4 Objectives

- Preserve uploaded and agent-collected evidence with SHA-256 hashes, manifests and an audit trail.
- Parse Linux authentication logs and Apache/nginx access logs into one timeline.
- Normalize timestamps to UTC only when the source or the investigator supplies enough information, and record the reason otherwise.
- Detect collection gaps and byte-range interruptions in agent-delivered data.
- Generate rule-based correlations between authentication and web events, each with a confidence level and an explanation.
- Provide a web interface for investigation management, timeline search, quality review and agent monitoring that needs no build step.
- Expose a documented REST API, PDF and CSV export, and a container-based deployment path.

1.5 Scope

In scope: single-tenant investigations; manual upload and collection by a Linux agent; the parsers and timestamp logic under backend/app; rule-based correlation; quality reporting; PDF and CSV export; and Docker-based deployment files.

Out of scope: multi-tenant isolation, user authentication and role-based access, cryptographic attestation of the original evidence, automated response or remediation, and any claim that a correlation proves a common actor or an attack. CyberTrace is a prototype intended for local or trusted-network use.

1.6 Development Phases

The system was built in phases, and the version history is visible in the migrations and the documentation.

Phase	Added
1	Investigations, evidence upload, SHA-256 preservation, Linux auth and web access parsers, timestamp normalization, searchable timeline, hash verification and reparse.
2	Optional Linux endpoint agent: enrollment, incremental collection, durable buffering, authenticated batch upload, heartbeats and the agent fleet dashboard.
3	Evidence-quality analysis: collection coverage and gaps, timestamp confidence, parser counters, manifests, non-destructive duplicate reconciliation and the evidence audit trail.
4.1	Investigation dashboard and unified timeline at /.
4.2	Rule-based cross-source correlation with explanations and review.
4.3	PDF investigation report and CSV exports.
5	Documentation set, synthetic scenarios, evaluation records and Docker deployment files.

1.7 Report Organization

Chapter	Contents
1	Introduction, problem statement, objectives and scope
2	Literature review and positioning against existing tools
3	System analysis: existing process, requirements, use cases and constraints
4	System design: architecture, deployment, data flow, data model, correlation, agent protocol and security

Chapter	Contents
5	Implementation: technology stack, backend, parsers, agent, frontend and deployment
6	Testing and evaluation
7	Results and discussion, with screenshots of the running system
8	Conclusion, limitations and future work
A to E	API inventory, database tables, test inventory, deployment notes and agent configuration reference

Chapter 2. Literature Review and Positioning

2.1 Digital Forensics Timeline Analysis

Timeline reconstruction is one of the basic activities in digital forensics. Events from different sources are placed in time order so that a sequence becomes visible, for example repeated failed logins followed by a successful one and then unusual web requests. NIST Special Publication 800-86 describes forensic work as a process of collection, examination, analysis and reporting, and stresses that the original data must be preserved and that each transformation applied to it must be documented [1]. CyberTrace follows that order: its pipeline preserves first, derives data second, and never writes back to the original.

2.2 Log Parsing and Normalization

Logs from different systems disagree about time. Traditional syslog (RFC 3164) lines have no year and no zone. Journald-style lines use ISO 8601 and usually include an offset. Apache and nginx access logs carry an explicit offset in the request timestamp. Reconciling these is not just a formatting problem, because choosing a zone or a year for a line that does not state one introduces an assumption that the final report may not reveal.

CyberTrace therefore stores, for every event, the original timestamp text, any zone information found, the resolved UTC value when one could be derived, and a status that says how the value was obtained or why it could not be. The OWASP Logging Cheat Sheet makes a related point that logs are only useful as evidence when the time and source of each record are reliable [2].

2.3 Evidence Integrity and Chain of Custody

A cryptographic hash taken when evidence is collected allows any later change to be detected. CyberTrace computes SHA-256 while the upload is streamed to disk, stores the digest with the file metadata, and can recompute it at any time. Each verification is recorded. Manifests list the preserved files with their hashes, sizes and collection details, and an audit table records preservation, parsing, download, verification, reparse, duplicate handling and manifest export.

A matching hash shows that the stored bytes are the same as at ingestion. It says nothing about whether the log was genuine before it reached the system, and the audit table is an application record, not a signed log. These limits are stated in the documentation and repeated in Chapter 8.

2.4 Related Tools

Several open-source tools address parts of the same problem space. They were studied as references rather than as competitors, since most are much larger in scope.

Tool	Focus	Relation to CyberTrace
Plaso (log2timeline)	Extracts timestamped events from many artifact types into a single "super timeline".	Far broader parser coverage. CyberTrace covers two log families but adds preservation records, timestamp status and quality reporting in the same workflow.
Timesketch	Collaborative exploration and annotation of timelines.	Strong analysis and collaboration features. CyberTrace concentrates on how evidence is collected and how reliable each timestamp is.
Velociraptor	Endpoint visibility and artifact collection at scale.	A full collection and response framework. The CyberTrace agent only reads configured log files and cannot run commands.

Tool	Focus	Relation to CyberTrace
Wazuh	Security monitoring, detection and log analysis platform.	Designed for continuous monitoring. CyberTrace is built for after-the-fact case work on a defined set of logs.

2.5 Gap Analysis

Capability	Typical ad-hoc handling	CyberTrace
Preservation and hashing	Manual copy, hash often not recorded	Automatic, with manifest and read-only storage
UTC normalization	Zone and year often assumed	Status recorded per event; unknown values stay flagged
Cross-source correlation	Manual searching	Rule-based, with confidence and explanation
Collection-gap detection	Usually none	Byte-range and heartbeat based coverage and gaps
Parsing failures	Lines silently skipped	Recorded as parsing issues with counters
Audit trail	None	Audit event for each evidence lifecycle action
Interface	Spreadsheets and terminals	Dependency-free browser dashboards and REST API

2.6 Positioning

CyberTrace is an auditable, small-scope timeline analyzer for labs, teaching and hackathon-scale investigations. It is not a SIEM, an endpoint detection product or a replacement for the tools above. Its contribution is the way it combines preservation, honest timestamp handling and evidence-quality reporting, and the fact that every correlation it produces can be traced to a rule and the events behind it.

Chapter 3. System Analysis

3.1 Existing Process and Its Limitations

In a typical manual investigation the analyst collects logs one system at a time, opens them with a text editor or command-line tools, compares timestamps by eye, builds a timeline in a spreadsheet or script, searches for shared addresses and user names, and writes the report by hand.

This has predictable weaknesses. It is slow and repetitive. Evidence handling varies from one analyst to the next. Differences in time zone lead to wrong ordering. Lines that fail to parse go unnoticed. A missing stretch of log is mistaken for inactivity. Relationships between sources are tracked in the analyst's head. The report may not mention integrity or uncertainty at all.

3.2 Proposed System

CyberTrace replaces the manual steps with one workflow and keeps the derived data separate from the evidence. The main capabilities are listed below.

- Investigation management and manual evidence upload.
- An optional Linux agent that collects new log bytes and delivers them in authenticated batches.
- Original evidence preservation with SHA-256 verification and manifests.
- Parsing of Linux authentication logs and Apache/nginx access logs.
- Timestamp normalization with status and uncertainty tracking.
- A unified timeline with search and filtering.
- Evidence-quality dashboard: coverage, gaps, parser counters, timestamp status, integrity, duplicates and audit trail.
- Explainable cross-source correlations that can be reviewed and dismissed.
- PDF report and CSV exports.

3.3 Functional Requirements

ID	Requirement
FR-1	Create, list and select investigations with a name and description.
FR-2	Upload log files (UTF-8 text, up to 50 MB) with source type, source name, hostname, timezone hint, default year, collection time, notes and an option to allow duplicates.
FR-3	Parse uploaded files into timeline events that keep both the original and the normalized timestamp.
FR-4	Preserve each file with SHA-256 and a manifest, and verify it on demand.
FR-5	Filter, paginate and inspect timeline events, including a detail view with the raw source line.
FR-6	Run correlations and review them with their confidence and rule identifier; dismiss false positives with a reason.
FR-7	Report coverage, gaps, parsing outcomes, timestamp status, duplicates, integrity and the audit trail.
FR-8	Enroll and monitor endpoint agents, accept batched collections and reconcile byte ranges.
FR-9	Export investigation data as CSV datasets and a PDF report.

3.4 Non-Functional Requirements

ID	Requirement
NFR-1	Reproducible schema through Alembic migrations with a single head (f42a00000001).
NFR-2	No frontend build step; the interface is plain HTML, CSS and JavaScript served by the API.
NFR-3	REST API documented through OpenAPI at /docs.
NFR-4	Container-based deployment using Docker Compose.
NFR-5	Auditable behaviour: every evidence lifecycle action is recorded.
NFR-6	Original evidence is never modified by parsing, reparsing or reporting.
NFR-7	Uncertainty is explicit: nothing is shown as exact unless the evidence supports it.

3.5 Use Cases

Actor	Use case
Investigator	Create an investigation, upload logs, explore the timeline, review quality indicators, run and review correlations, and export reports.
Endpoint agent	Enroll with the server, send heartbeats, upload hashed batches and report source health.
Reviewer	Verify hashes, inspect manifests and read the audit trail.
Administrator	Set the enrollment token, enable or disable agent registrations, and run migrations.

3.6 Investigation Workflow

The documented workflow for a case runs through the following steps. The software supports each one, but the judgement involved, such as which correlations matter, remains with the investigator.

1. Define the scope of the incident and create an investigation.
2. Collect evidence from authorized systems by upload or through the agent.
3. Preserve the files and record their SHA-256 hashes.
4. Parse the logs and normalize timestamps without guessing unknown values.
5. Review collection gaps and parsing failures.
6. Verify evidence integrity.
7. Reconstruct the timeline.
8. Review cross-system correlations and read their explanations.
9. Dismiss false positives with a recorded reason.
10. Generate and archive a report.
11. Document limitations and unresolved questions.

3.7 Constraints and Assumptions

- PostgreSQL 14 or newer is required. SQLite is not supported for the server.
- Logs must be UTF-8 text. Binary evidence is rejected.
- The correlation time window defaults to 1800 seconds and can be set between 60 and 86400 seconds.
- Agent registration requires AGENT_ENROLLMENT_TOKEN. If it is not set, registration is disabled and the server returns 503.
- The dashboard and API have no login. They are meant for localhost or a trusted network.

- The agent targets Linux with systemd and Python 3.11 or newer.

Chapter 4. System Design

4.1 Architecture Overview

CyberTrace is organised in layers. A static single-page interface is served by a FastAPI application. The routers in that application call a service layer that holds the ingestion, parsing, correlation, quality, reporting and agent logic. The services use SQLAlchemy to reach PostgreSQL and write evidence files to a separate filesystem store. Figure 4.1 shows the clients, the API, the services and the two data stores.

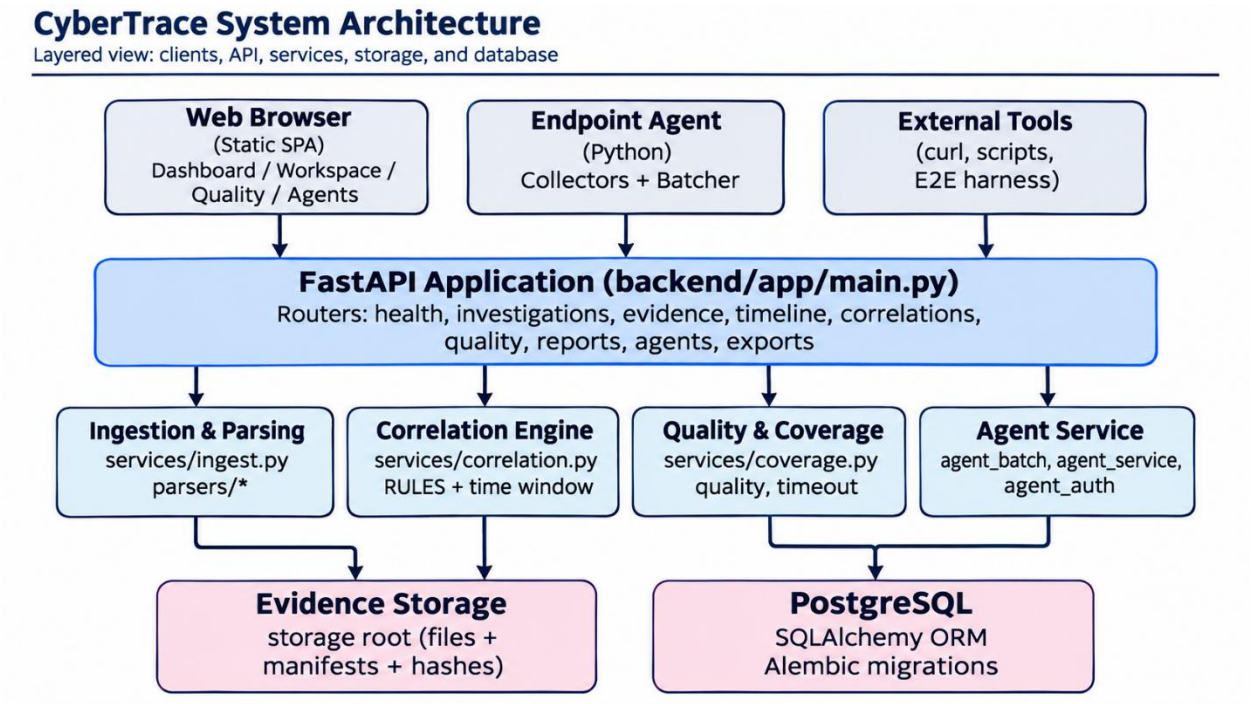


Figure 4.1 System architecture: layers and dependencies

Three kinds of client talk to the same API. The browser uses the dashboards. The endpoint agent uses a small set of authenticated routes. Scripts and the end-to-end harness use the REST API directly. Keeping a single API in front of everything means that evidence handling and audit recording happen in one place, whichever client is used.

4.2 Deployment Architecture

Figure 4.2 shows how the pieces are placed when the system is deployed. The investigator works from a browser. A monitored Linux host runs the agent, which reads log files and keeps its own state in SQLite. Both reach the server over HTTPS through a reverse proxy such as nginx or Caddy, since the agent refuses plain HTTP except for loopback labs. The server side is a Docker Compose stack with two containers: the API, which runs as a non-root user, and PostgreSQL. Preserved evidence is written to a storage directory outside the database.

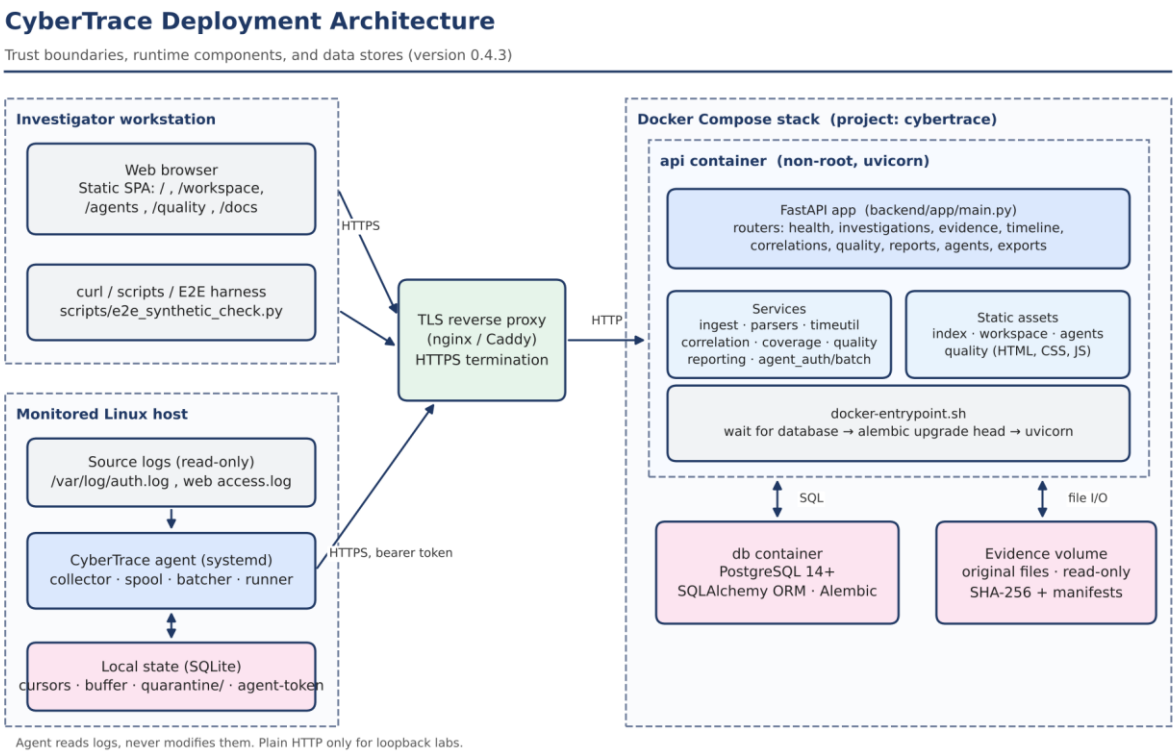


Figure 4.2 Deployment architecture and trust boundaries

At startup the API container runs docker-entrypoint.sh, which waits for the database, applies the Alembic migrations and then starts uvicorn. This is why a fresh database does not need a separate setup step. The trust boundary that matters most is the one between the monitored host and the server: the agent holds only its own bearer token, and the server stores only the SHA-256 hash of that token.

4.3 Backend Component Architecture

Inside the API container the code is split so that each concern can be tested alone. Figure 4.3 shows the four layers and the modules in each. Routers do request handling and validation through Pydantic schemas. Services contain the logic. The ORM models and the evidence store sit below them.

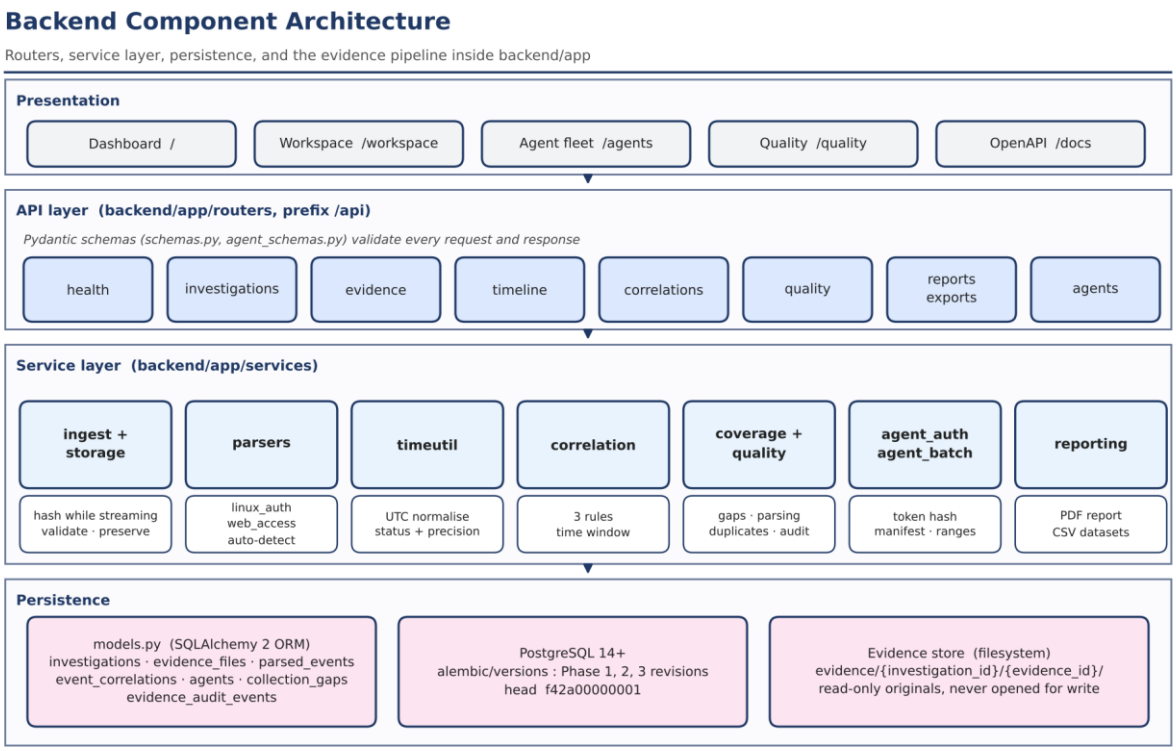


Figure 4.3 Backend component architecture

Module	Responsibility
ingest, storage	Validate an upload, stream it to disk while hashing, move it to a generated evidence path, mark it read-only and record the metadata.
parsers	Detect the log family from the first 100 non-blank lines (or use an explicit source type) and turn each line into an event or a parsing issue.
timeutil	Resolve timestamps to UTC where the evidence allows, and assign the timestamp status and precision.
correlation	Apply the correlation rules inside the configured time window and assign confidence.
coverage, quality	Derive source coverage, collection gaps, parser counters, duplicate groups, integrity state and audit views.
agent_auth, agent_batch	Check enrollment and bearer tokens, validate batch manifests, store batch items and make them idempotent.
reporting	Produce the PDF report and the CSV datasets from stored data.

4.4 Evidence Processing Data Flow

Every evidence file follows the same path, whether it arrives by upload or in an agent batch: receive, hash, record the manifest, parse, normalize, persist, correlate and present. Figure 4.4 shows the stages. Only the first three touch the original bytes. Everything after that works on derived rows that can be rebuilt from the preserved file.

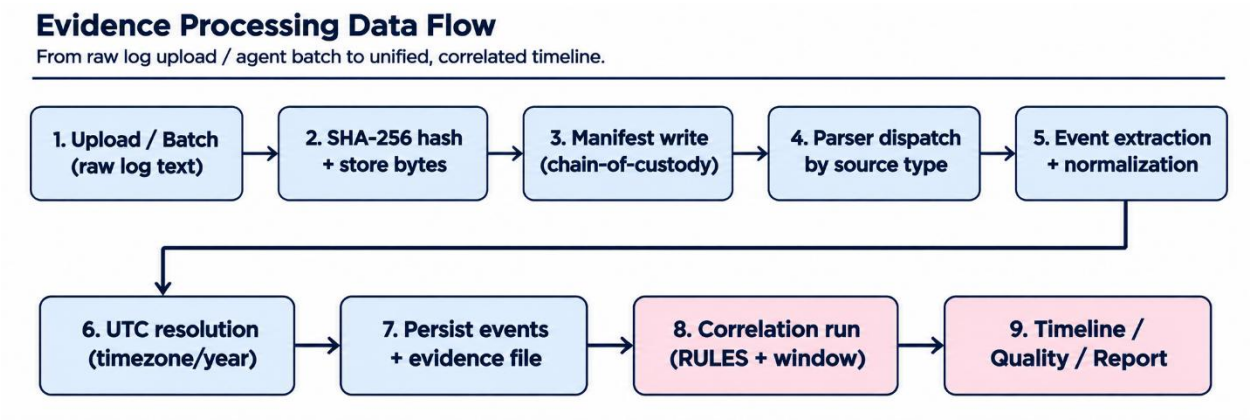


Figure 4.4 Evidence processing data flow

This arrangement gives the design its main property. If a parser is fixed or a timezone hint is supplied later, the events can be regenerated from the original file, after the hash has been verified, without touching the evidence itself.

4.5 Data Model

The data is stored in PostgreSQL through SQLAlchemy models, with schema changes managed by Alembic. Figure 4.5 shows the main entities in simplified form and Table 4.1 lists the tables.

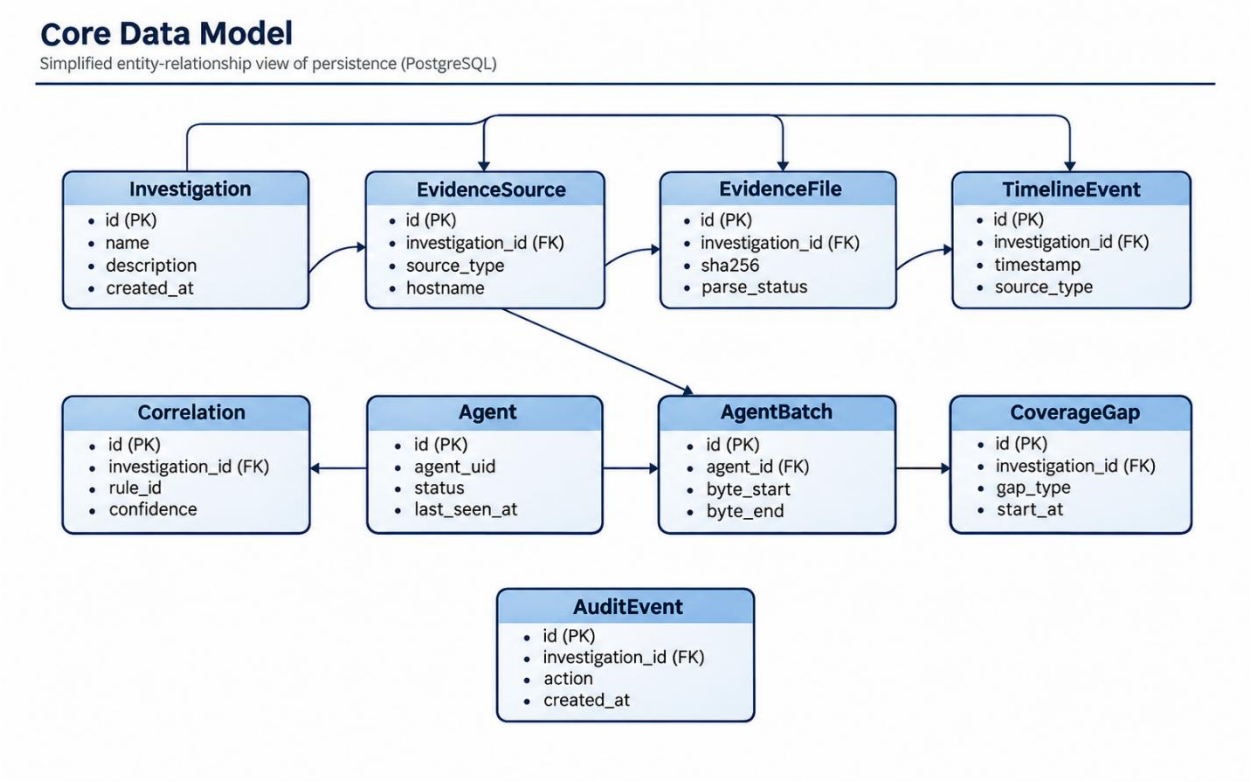


Figure 4.5 Core data model

Table 4.1 Main database tables

Table	Purpose
investigations	The investigation container: name, description and creation time.
evidence_sources	Source metadata for each investigation, such as type and hostname.
evidence_files	One row per preserved file: SHA-256, size, original filename, collection time, storage path and parse status.

Table	Purpose
parsed_events	Normalized timeline events with original and UTC timestamps, status and extracted identifiers.
parsing_results, parsing_issues	Per-file parsing outcome and the malformed, unsupported or flagged records.
event_correlations, event_correlation_members	Correlation groups with rule, confidence and explanation, and the events that belong to each.
reconciliation_runs	Records of duplicate reconciliation.
agents, agent_sources	Registered agents and the log sources each one is configured to collect.
agent_batches, agent_batch_items	Uploaded batches and the hashed segments inside them, with byte ranges.
source_collection_windows, collection_gaps	Collection windows from heartbeats and batches, and the observed or inferred gaps derived from them.
evidence_audit_events	The evidence lifecycle audit trail.

The relationships follow the way a case is built. An investigation owns its sources, files, events, correlations and audit events. A file has parsing results, events, hash records and verifications. An event points back to its file and source. A correlation points to several events. A batch contains segments, and the collection windows from those segments are what the coverage calculation uses.

4.6 Timestamp Handling

Timestamp handling is the part of the design where the system is most careful. The rule is simple: a UTC value is produced only when the evidence supports it.

Situation	Result
The line carries an offset (ISO 8601, Apache/nginx)	Converted to UTC; status normalized.
The line has no zone but the investigator supplies timezone_hint	Converted using the hint; the hint is recorded.
The line has no year (classic syslog) and no default_year is supplied	Not resolved; status missing_year.
The line has no zone and no hint	Not resolved; status missing_timezone.
The text cannot be interpreted	Not resolved; the record is listed as a parsing issue (unknown_timestamp).

Events whose time could not be resolved are kept and are placed last in ascending order. The timeline API can exclude them with include_unknown_time=false or isolate one status with timestamp_status=. A source-level clock_offset_seconds can be configured by the investigator and is then applied and recorded as applied. An offset measured by an agent against server time is stored for visibility but is never applied automatically, because the system does not estimate clock skew on its own.

4.7 Correlation Heuristic

A correlation links two events from different evidence sources that share an identifier and are close in time. The default window is 1800 seconds and can be set between 60 and 86400 seconds. Figure 4.6 shows the workflow.

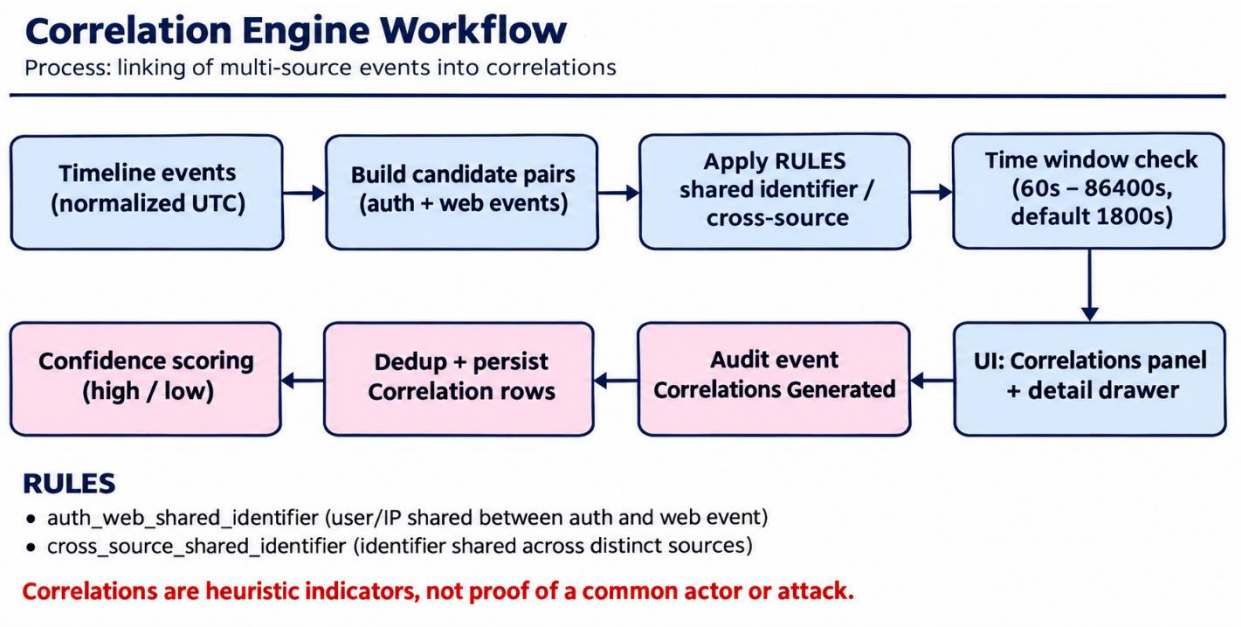


Figure 4.6 Correlation engine workflow

Rule	Matches
auth_web_shared_identifier	An authentication event and a web-access event from different sources that share a source IP or a user name.
cross_source_shared_identifier	Events from different sources that share a source IP or a user name.
cross_source_shared_hostname	Events from different sources that share a hostname. Reported as low confidence.

Matching values are compared without regard to case. When both events have a normalized UTC timestamp, the difference must lie within the window. If either timestamp is missing, only a strong identifier (IP or user name) can create a correlation, and the timestamp is not used as ordering evidence. Hostname-only matches are always low confidence. Duplicate events are kept and may appear in correlations.

Confidence	Condition
high	A strong shared identifier across different source types, both timestamps normalized, and the events within 5 minutes of each other.
medium	A strong shared identifier with normalized timestamps inside the configured window.
low	A shared identifier with missing or uncertain timestamps, or a hostname-only match. Treated as a possible link only.

Every correlation records its rule, its confidence and a plain-language explanation, and carries the timestamp uncertainty of its events. An investigator can dismiss a correlation with a reason. A correlation is a lead, not proof that the same actor was responsible for both events.

4.8 Endpoint Agent Protocol

The agent registers with an enrollment token, sends heartbeats, collects new log bytes, hashes and batches them with byte ranges, and uploads them. The server ingests the batch through the normal pipeline and reconciles the byte ranges to find gaps. Figure 4.7 shows the sequence.

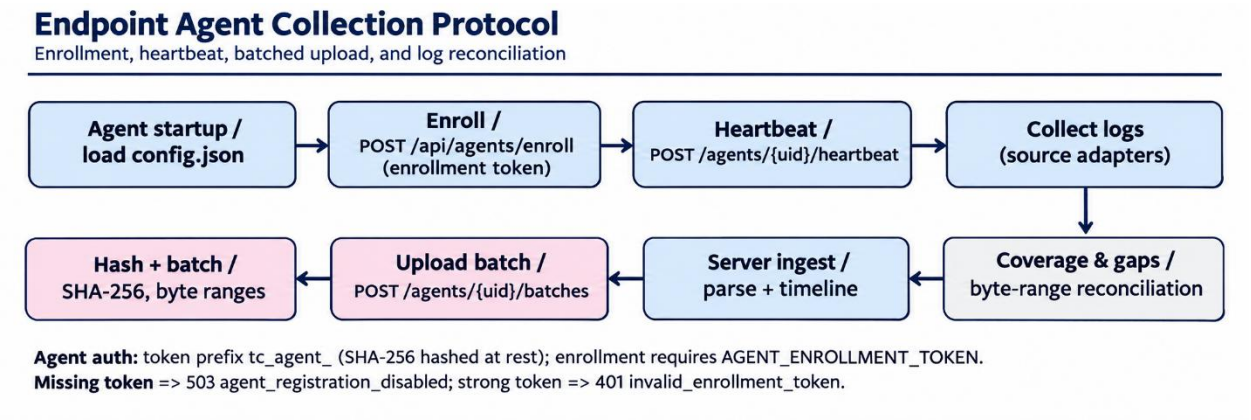


Figure 4.7 Endpoint agent collection protocol

The figure labels the routes in a simplified form. In the implementation the agent registers with POST /api/agents/register using the enrollment token, and afterwards identifies itself with its bearer token on POST /api/agents/heartbeat and POST /api/agents/batches, so the agent identifier does not appear in the path. The behaviour of the agent between the steps is what keeps the data safe:

- Each source is a regular file. The agent stores a byte cursor together with the file's device and inode.
- New content is read in bounded segments, written to a spool file, hashed, and only then added to a durable batch. A crash cannot move the cursor past bytes that were never queued.
- Rotation or truncation is detected by a changed inode or a smaller file size, and the replacement is read from the beginning.
- Batches are retried with bounded exponential backoff. Server, TLS and network failures do not discard data.
- The local buffer is limited by max_buffer_bytes. When it is full, collection stops and the source reports buffer_full.
- Batches that the server rejects permanently, or that are too large, are moved to a quarantine/ folder with a QUARANTINE.txt explaining why, so the queue cannot loop forever.

4.9 Evidence Preservation and Integrity

1. The upload is streamed to a temporary file while SHA-256 is computed.
2. Empty files, files containing NUL bytes, files with a disallowed extension and files over the size limit are rejected.
3. The filename is sanitized and the file is stored under a generated directory of the form evidence/{investigation_id}/{evidence_id}/.
4. The stored file is made read-only. PostgreSQL stores the hash, size, original filename, collection time, source and relative path.
5. A file whose SHA-256 already exists in the same investigation is rejected with HTTP 409 unless allow_duplicate=true.
6. POST /api/evidence/{id}/verify recomputes the hash and records the result.
7. POST /api/evidence/{id}/reparse verifies the hash first and then replaces only the derived parsing rows. A mismatch refuses the reparse.
8. A canonical manifest, with its own SHA-256, can be exported for a whole investigation or for one file.

Read-only permissions are best-effort on Windows. The application itself never opens an evidence file for writing after it has been preserved.

4.10 Collection Coverage and Gaps

Coverage is calculated from agent heartbeats, source status reports and byte-range collection windows. Each source is classified as reporting, idle, not_reporting, error, disabled or unknown. Sources that arrive by manual upload, or that have no agent telemetry, stay unknown. They are never counted as healthy.

A collection gap has a scope, a reason, a basis and a status. The reasons include missed heartbeat cycles, reported collection failures, byte-range gaps between segments and silence from a source that was reporting before. The basis is either observed, when it comes from an explicit report, or inferred, when the system expected collection and did not see it. An inferred gap is a quality warning. It does not show that the host produced no activity, and a cycle with zero bytes does not either.

4.11 Security Design

- Agent tokens carry the prefix ct_agent_ and only their SHA-256 hash is stored. A disabled agent's token is rejected immediately.
- Enrollment needs AGENT_ENROLLMENT_TOKEN. If it is not set the server returns 503 agent_registration_disabled; a wrong value returns 401 invalid_enrollment_token.
- Uploads are limited to 50 MB and validated as UTF-8 text. Uploaded content is never executed or passed to a shell.
- Raw log lines and agent error messages are escaped when shown in the dashboards.
- CORS and trusted hosts are configurable. Database credentials come from environment variables and none are embedded in the code.
- The enrollment token entered in the dashboard is held in page memory only and is not persisted.

Several protections are deliberately absent and are listed as limitations in Chapter 8: there is no investigator login, no role-based authorization, no malware scanning and no encryption of evidence at rest by the application.

Chapter 5. Implementation

5.1 Technology Stack

Layer	Technology
Backend	Python 3.11+, FastAPI, SQLAlchemy 2, Alembic, Pydantic, psycopg
Database	PostgreSQL 14+
Frontend	Plain JavaScript, HTML and CSS with no build step, served by FastAPI
Agent	Python with no third-party dependencies; SQLite for local state
Deployment	Docker, Docker Compose, .env configuration
Testing and quality	pytest for backend and agent, Ruff for linting, node --check for JavaScript syntax, a synthetic end-to-end script
Reports	PDF and CSV generated on demand

The development and verification machine ran Windows with PostgreSQL 18.6 and Python 3.13. The agent and the supplied systemd unit target Linux.

5.2 Repository Layout

```
agent/
  cybertrace_agent/   client, collector, config, runner, state, cli
  tests/              agent pytest suite
  systemd/            hardened service unit example
  config.example.json agent configuration template
backend/
  app/
    main.py           application factory, static pages, startup checks
    models.py         SQLAlchemy schema
    schemas.py        investigator API models
    agent_schemas.py  agent API models
    routers/          API endpoints
    services/         ingest, storage, parsers, time handling, correlation,
                      coverage, quality, reporting, agent authentication
    static/           dashboard, workspace, fleet and quality pages
  alembic/            migration environment and revisions
  tests/              backend pytest suite and fixtures
  Dockerfile, docker-entrypoint.sh, requirements*.txt, pytest.ini
samples/              three synthetic incident scenarios
docs/                 installation, architecture, API, security and other notes
reports/              setup and verification records
scripts/              e2e_synthetic_check.py
docker-compose.yml, .env.example, README.md
```

5.3 Backend

main.py builds the application, registers the routers (health, investigations, evidence, timeline, correlations, quality, reports, agents and exports), mounts the static pages and checks the database and storage at startup. config.py reads settings from the environment. constants.py holds the enumerations for source types, timestamp statuses and gap reasons so that the same strings are used in the database, the API and the interface.

Request and response shapes are defined with Pydantic in schemas.py and agent_schemas.py. Serialization of events and evidence into API responses is kept in serializers.py, which keeps the routers short. The health endpoint reports database connectivity, schema version, storage availability and application version.

5.4 Parsers and Timestamp Resolution

Two log families are supported. Linux authentication logs may use the RFC 3164 form (Mon DD HH:MM:SS host program[pid]: message) or an ISO or journald-style timestamp. Apache and nginx access logs are read in common and combined format, which supplies the client address, request line, status code and, for combined format, the referrer and user agent.

When the source type is auto, the first 100 non-blank lines are sampled to decide the family. Very unusual mixed files can be misclassified, and the upload form lets the investigator set the type explicitly. Lines that cannot be parsed, are unsupported, or carry a flagged timestamp are not dropped. They become parsing issues with a type and severity. The number of issues stored is capped, and the issues suppressed by the cap are counted so that the totals stay correct.

Timestamp resolution follows the rules in Section 4.6. Event ordering ties, timestamp precision and any applied or merely measured clock offset are recorded on the event so that the timeline can be reproduced.

5.5 Evidence Storage and Integrity

Evidence bytes are written below a storage root, outside the database, under evidence/{investigation_id}/{evidence_id}/. The SHA-256 digest is computed on the stream, so a large file is never read into memory twice. After the move the file is made read-only. Verification recomputes the digest and compares it with the stored baseline, and each result is recorded in a verification table and in the audit trail.

The application does not delete evidence files. If a database failure happens after a file has been preserved, an orphaned but valid file can remain; the error message reports its relative path so that it can be reconciled.

Duplicate detection works on parsed events. Events with the same fingerprint are linked to a canonical event rather than removed. Reconciliation can be run again to rebuild the links. Because two legitimate log lines can be identical, canonical choices should be reviewed before unique-event counts are relied on.

5.6 Frontend

The interface is a set of static pages in backend/app/static, each with its own script and no framework or build step. The API base is /api.

Route	Page	Purpose
/	Investigation dashboard	Summary counters, collection coverage, parsing failures, timestamp uncertainty, integrity and the unified timeline on one screen.
/workspace	Evidence workspace	Import evidence, list collected files, search the timeline and open event details.
/agents	Agent fleet	Agent health (online, stale, never seen, disabled), sources, batches and errors.
/quality	Quality dashboard	Coverage, gaps, parsing, timestamps, integrity, duplicates and audit trail for one investigation.
/docs	OpenAPI	Interactive API documentation generated by FastAPI.

The quality page keeps the selected investigation in localStorage under cybertrace.currentInvestigation. All text that comes from logs is inserted as text, not as markup.

5.7 Endpoint Agent

The agent is a standalone Python package in agent/cybertrace_agent with no third-party dependencies. The modules are config (strict JSON configuration that rejects unknown keys), collector (incremental reading and rotation detection), state (SQLite cursors, buffer and identity), client (HTTPS calls and error mapping), runner (heartbeat, collection and flush loops) and cli.

It runs under systemd as a dedicated cybertrace account with read-only access to /var/log. Read access to the logs is granted with group membership (for example adm) or POSIX ACLs. The enrollment token is read from a separate file only when enrollment is needed and is never written to the state directory; the per-agent bearer token is stored in agent-token with mode 0600. The most important settings are listed in Appendix E.

The agent is a collection component only. It does not execute commands, inspect processes or collect files other than those it is configured to read.

5.8 Reporting and Export

Reports are generated on demand from stored data, so they always reflect the current state of the investigation. The PDF contains evidence hashes, collection gaps, parsing failures, timestamp uncertainty, integrity status and the explanation for each correlation. Six CSV datasets can be exported: timeline, evidence, coverage, gaps, parsing and correlations. Generation is synchronous, which is acceptable for small cases and is listed as a limitation.

```
GET /api/investigations/{id}/report.pdf
GET /api/investigations/{id}/export/timeline.csv
GET /api/investigations/{id}/export/evidence.csv
GET /api/investigations/{id}/export/coverage.csv
GET /api/investigations/{id}/export/gaps.csv
GET /api/investigations/{id}/export/parsing.csv
GET /api/investigations/{id}/export/correlations.csv
```

5.9 Deployment

The repository includes a Compose file for the API and database, a backend Dockerfile that runs as a non-root user, and an entrypoint script. The agent is installed separately on each monitored host. Details are in Appendix D. The Docker files were written and reviewed, but the container build was not executed on the machine used for verification (see Section 6.6).

5.10 Key Code Excerpts

Correlation rules (services/correlation.py):

```
RULES = {
    "auth_web_shared_identifier": ...,
    "cross_source_shared_identifier": ...,
    "cross_source_shared_hostname": ...,   # low confidence
}
TIME_WINDOW: min 60 s, default 1800 s, max 86400 s
```

Agent authentication (services/agent_auth.py): a missing enrollment token on the server gives 503 agent_registration_disabled; a wrong token gives 401 invalid_enrollment_token; a valid bearer token is hashed with SHA-256 and compared with the stored hash.

Chapter 6. Testing and Evaluation

6.1 Test Strategy

Testing is done in three layers. Backend unit and integration tests run with pytest against a dedicated PostgreSQL test database. Agent tests run against a local loopback HTTP fixture and need neither PostgreSQL nor a running server. A synthetic end-to-end script then exercises upload, parsing, the timeline, correlation, quality reporting and the agent flow against a live server. Linting and JavaScript syntax checks run alongside.

All tests use synthetic logs. The backend test configuration refuses to run unless the database name ends in _test, and it recreates the known tables for each run. Uploads in tests go to a temporary storage root and never to the real storage directory. The figures reported here were taken from actual command output, and no benchmark claims are made.

Table 6.1 Commands used for verification

Area	Command
Backend tests	python -m pytest -q (from backend)
Backend lint	python -m ruff check app tests
Compilation check	python -m compileall -q app tests
Agent tests	python -m pytest -q (from agent)
JavaScript syntax	node --check on app.js, dashboard.js, agents.js and quality.js
End-to-end	scripts/e2e_synthetic_check.py against a running server

6.2 Backend Tests

The backend suite has 81 passing tests across 10 test modules (79 test functions, with some parametrized) and a shared conftest.py. Ruff reports no problems. The suite covers:

- valid and malformed parsing, timestamp uncertainty and daylight-saving transitions;
- hashing, verification and tamper detection, duplicates, reparsing and upload validation;
- database persistence, timeline filtering and ordering, and the three synthetic scenarios;
- agent enrollment and token rotation, source inventory, heartbeat errors, agent management, batch hashing, parsing and idempotency, and manifest validation failures;
- coverage and gap derivation, timestamp uncertainty and clock offsets, duplicate linking and reconciliation, audit events and manifests;
- correlation rules, report generation, the investigation summary and sources endpoints, and all four dashboards.

Table 6.2 Backend test modules

Test module	Lines
test_api_agents.py	493
test_evidence_uplod.py	382
test_quality.py	352
test_correlation.py	174
test_reports.py	139

Test module	Lines
test_timeline.py	134
test_api_investigations_health.py	123
test_parsers.py	110
test_time_handling.py	63
test_samples.py	63
conftest.py	167
Total	2,200

6.3 Agent Tests

The agent suite has 10 passing tests. They cover configuration validation, incremental collection and rotation detection, cursor persistence, buffering and delivery, retry and backoff, quarantine, mapping of authentication errors, state identity conflicts, and the collection and heartbeat fields added in Phase 3.

6.4 Synthetic End-to-End Check

scripts/e2e_synthetic_check.py runs 31 checks against a live server and a freshly migrated database with an isolated storage root. All 31 passed.

6.5 Synthetic Scenarios

Three scenarios in the samples/ directory serve as the evaluation datasets. Each generated event was compared with what the system recovered.

Table 6.3 Evaluation scenarios

Scenario	Content	Generated	Recovered
1. SSH brute force and web probing	7 authentication events and 5 web events with a shared source IP	12	12
2. Web intrusion indicators	4 authentication events and 5 web events, including SQL injection and path-traversal probes	9	9
3. Timestamp uncertainty	2 authentication events and 1 web event; needs timezone_hint=Asia/Kolkata and default_year=2026	3	3 (with metadata)

Without the timezone and year metadata, Scenario 3 produces events whose timestamps stay unresolved and flagged, which is the intended behaviour. Three further procedures are documented for manual and demonstration use:

- **Agent disconnection and recovery.** Stop the agent, generate events, restart it, and check coverage gaps, buffered delivery, duplicate handling and the recovered events.
- **Timestamp uncertainty.** Run Scenario 3 without metadata and confirm that the timestamps are flagged as unknown.
- **Integrity and parsing failure.** Modify a stored evidence copy after hashing and confirm the mismatch; include malformed and unsupported records and confirm they appear as parsing issues.

The tampering test detected the hash mismatch, and the PDF and CSV endpoints returned valid files.

6.6 Quality Metrics on the Demonstration Dataset

A demonstration investigation named "Synthetic Incident 2026-10-31 - SSH Brute Force and Web Intrusion" was seeded and used for the screenshots in Chapter 7.

Table 6.4 Demonstration dataset metrics

Metric	Value
Evidence files	8
Timeline events	29 (linux_auth 18, web_access 11)
Correlations	52 (40 high, 12 low)
Correlations by rule	42 auth_web_shared_identifier, 10 cross_source_shared_identifier
Timestamp status	26 normalized, 2 missing_year, 1 missing_timezone
Parsing issues	3 (unknown_timestamp)
Hash verification	2 verified, 6 unverified, 0 mismatched
Coverage gaps	1 (closed byte-range gap)
Audit events	27
Agent	lab-edge-01.cybertrace.demo (online, 2 batches, 5 events)

The three parsing issues match the three events whose timestamps could not be resolved (two without a year, one without a zone), and the 29 events are consistent with the 18 authentication and 11 web events listed.

6.7 Docker Verification

Docker Engine was not available on the machine used for verification, so the image build and the container runtime were not executed there. The Compose file, backend Dockerfile, entrypoint and .env.example are included, and the verification that could be run without Docker (a fresh-database migration chain, the 31-check end-to-end run against a newly migrated database, restart persistence, and the lint and test suites on the final file state) is recorded in reports/DOCKER_SETUP_AND_VERIFICATION.md. No production-readiness claim is made.

6.8 Performance Notes

The backend uses synchronous SQLAlchemy sessions per request. The timeline is paginated and correlation runs on demand. Free-text search uses ILIKE, which suits small cases and not large full-text workloads. Report generation is synchronous. No load test or large-dataset benchmark has been run, so the system should be considered suitable for small to medium investigations only.

6.9 Interpretation of Results

The results show that the system behaves as designed on its automated tests and synthetic datasets. They do not show how it performs on large real-world investigations, and they do not measure detection accuracy against real attacks. The correlation counts in Table 6.4 describe how many candidate links the rules produced on a small seeded dataset, not how many of them are meaningful.

Chapter 7. Results and Discussion

This chapter shows the running system using the demonstration dataset described in Section 6.6. The screenshots are unmodified captures of the browser interface.

7.1 Dashboard

The dashboard selects the first investigation automatically and shows summary counters for evidence, events, sources, collection gaps, uncertain timestamps, verified hashes and parsing issues. This is the first view an investigator sees, and it puts the quality indicators next to the event counts rather than on a separate page.



Figure 7.1 Dashboard summary metrics

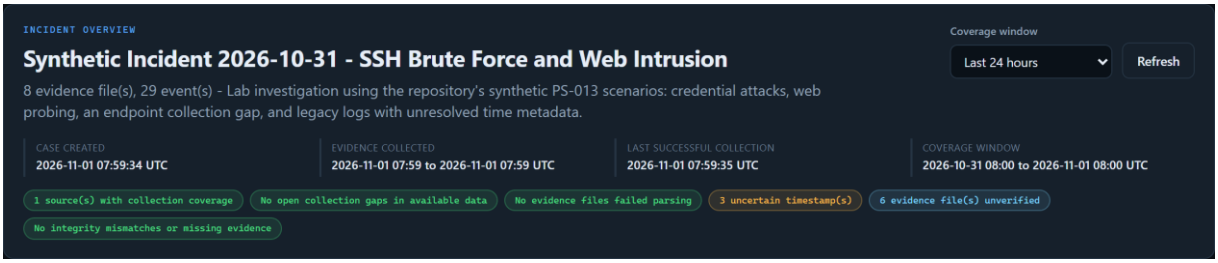


Figure 7.2 Dashboard overview panel

7.2 Sources, Gaps and Integrity

Figure 7.3 lists the evidence sources of the investigation with their coverage state. Figure 7.4 shows the collection gap recorded for the agent source, which in this dataset is a byte-range gap that was later closed. Figure 7.5 shows the integrity view, where the files that have been verified are separated from those that have not yet been checked.

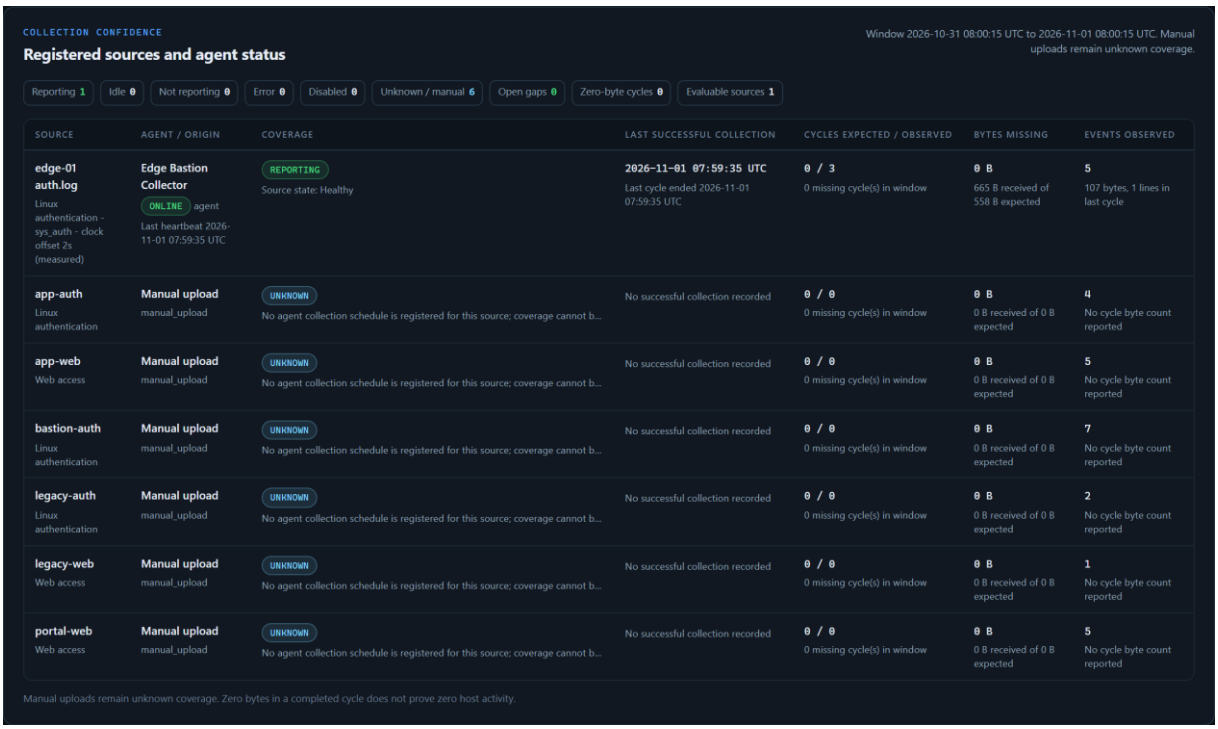


Figure 7.3 Evidence sources and collection gaps

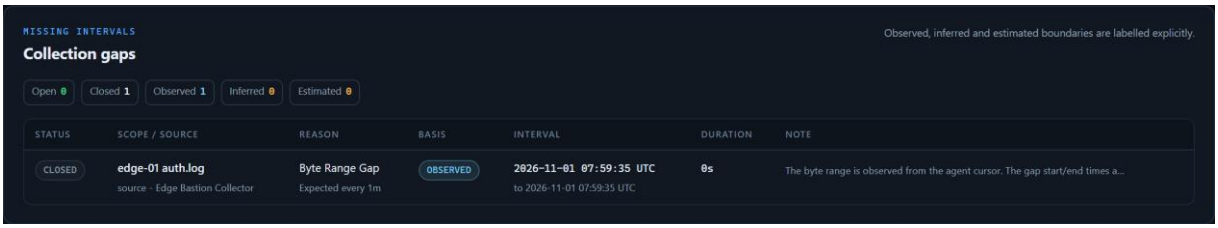


Figure 7.4 Coverage gaps

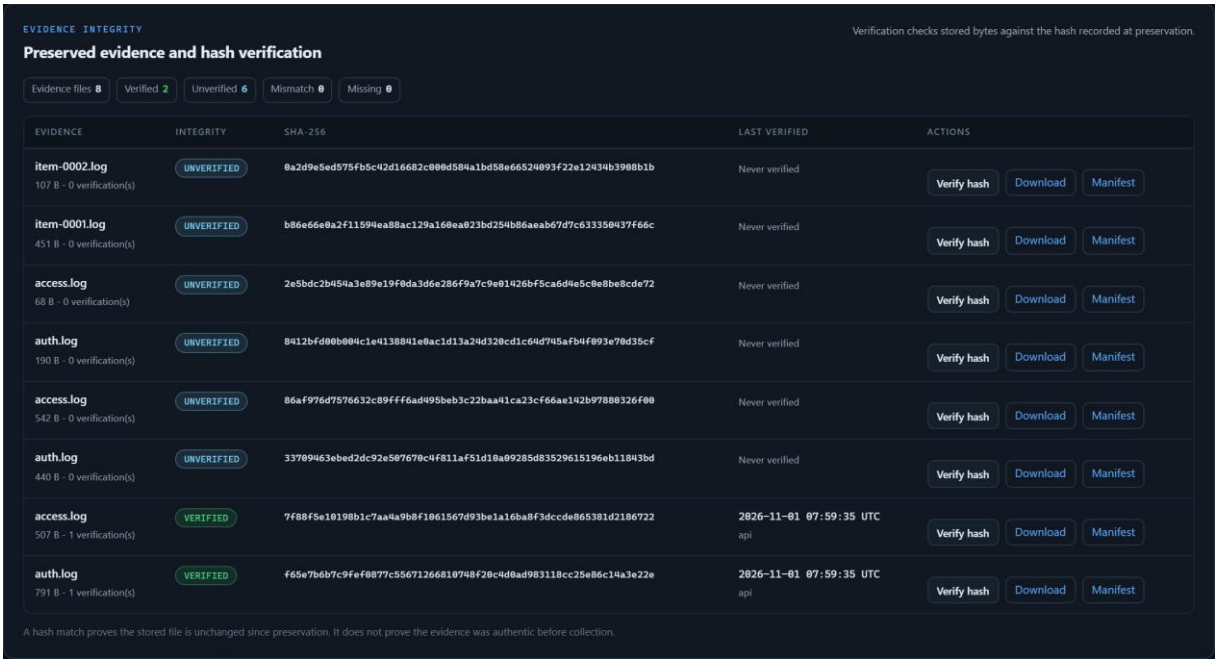


Figure 7.5 Integrity and hash verification

7.3 Correlations

The correlations panel lists the links the rules found. Each row carries its rule, its confidence and the events involved, so the reasoning can be checked against the underlying log lines. On this dataset the rules produced 52 correlations, 40 of them high confidence and 12 low.

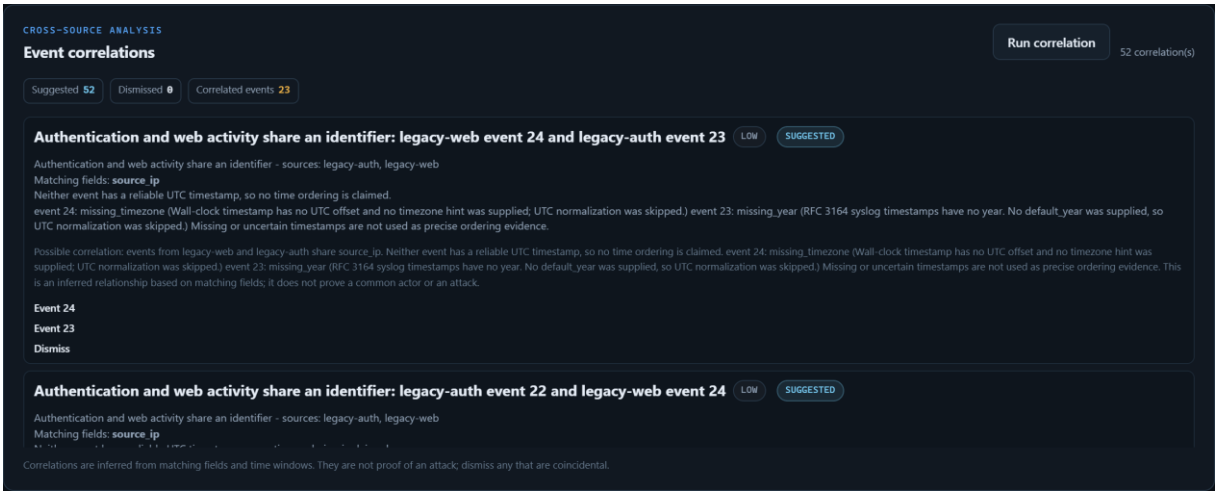


Figure 7.6 Correlations panel

7.4 Timeline and Event Detail

The unified timeline merges events from all sources into one searchable list that can be filtered by time, source, event type, host, user, source IP, HTTP status and free text. Selecting an event opens a detail drawer with the normalized fields, the original timestamp text, the timestamp status and the raw source line.

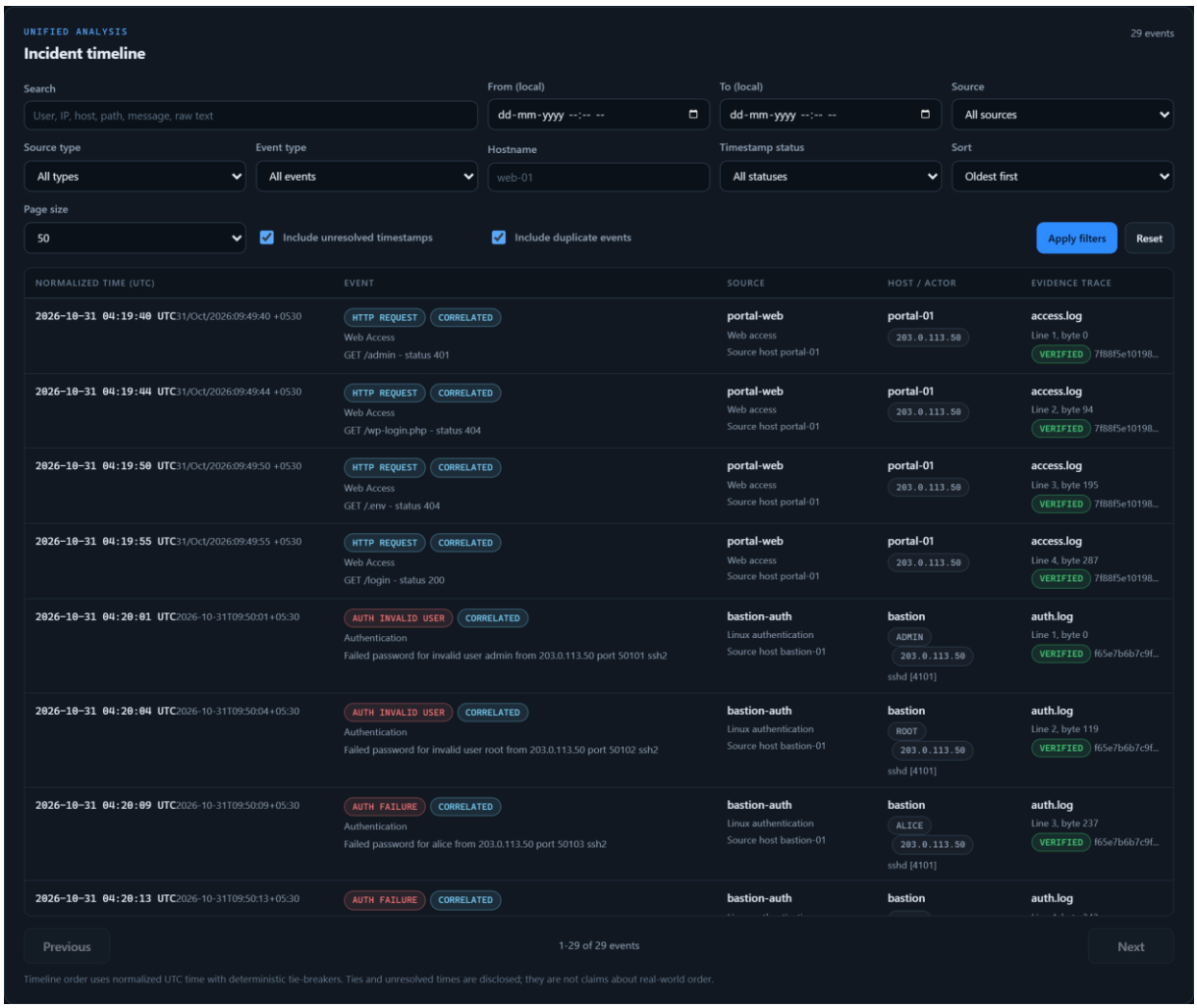


Figure 7.7 Unified timeline

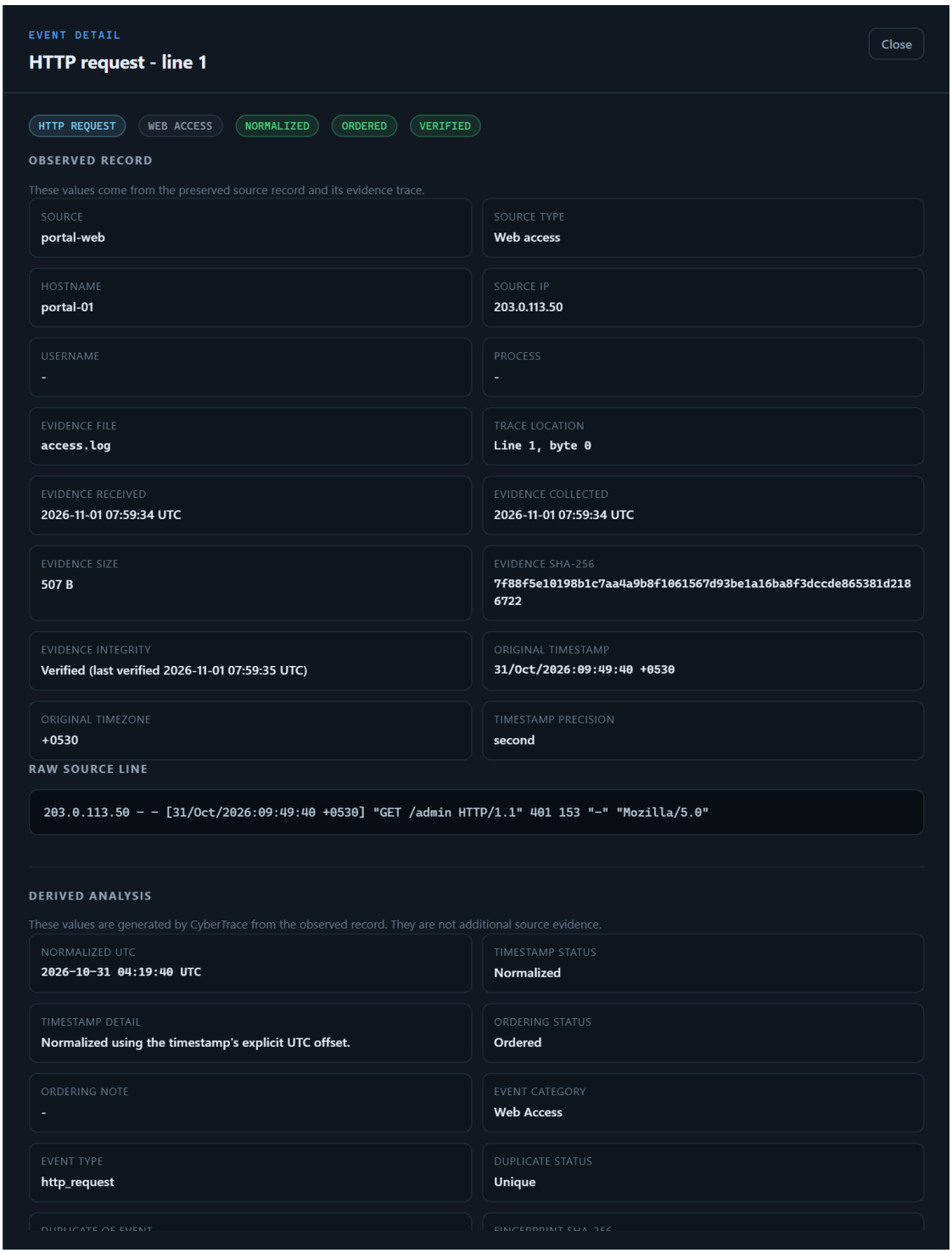


Figure 7.8 Event detail drawer

7.5 Quality Page

The quality page is where the evidence is judged rather than read. It starts with summary counters (Figure 7.9), then shows source coverage and gaps (Figure 7.10), the UTC normalization status of events (Figure 7.11), the preservation and integrity checks (Figure 7.12) and the evidence lifecycle audit (Figure 7.13).

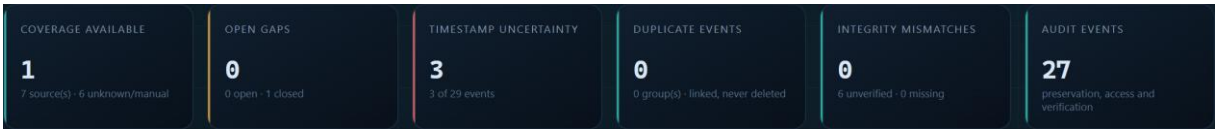


Figure 7.9 Quality summary metrics

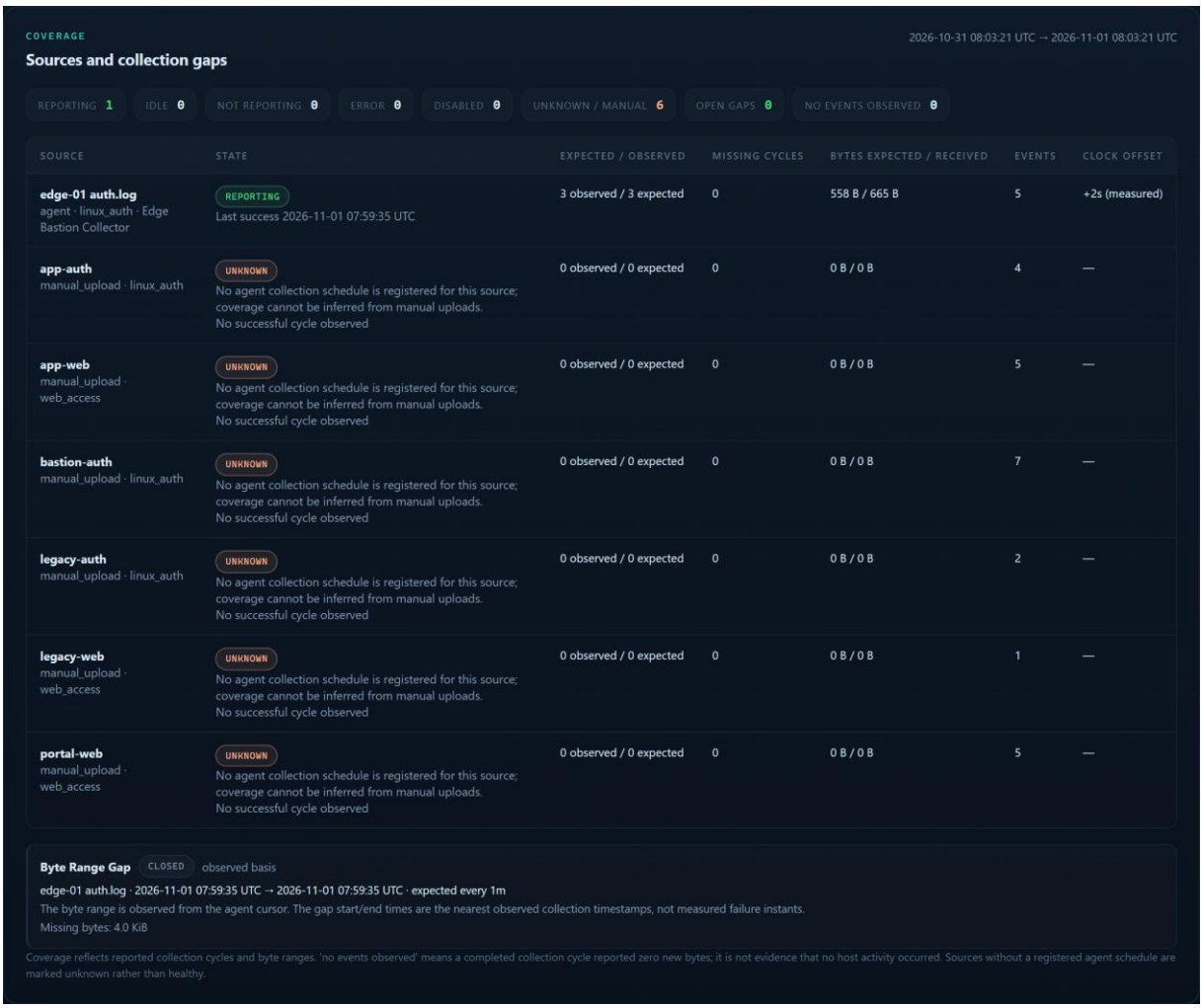


Figure 7.10 Sources and collection gaps

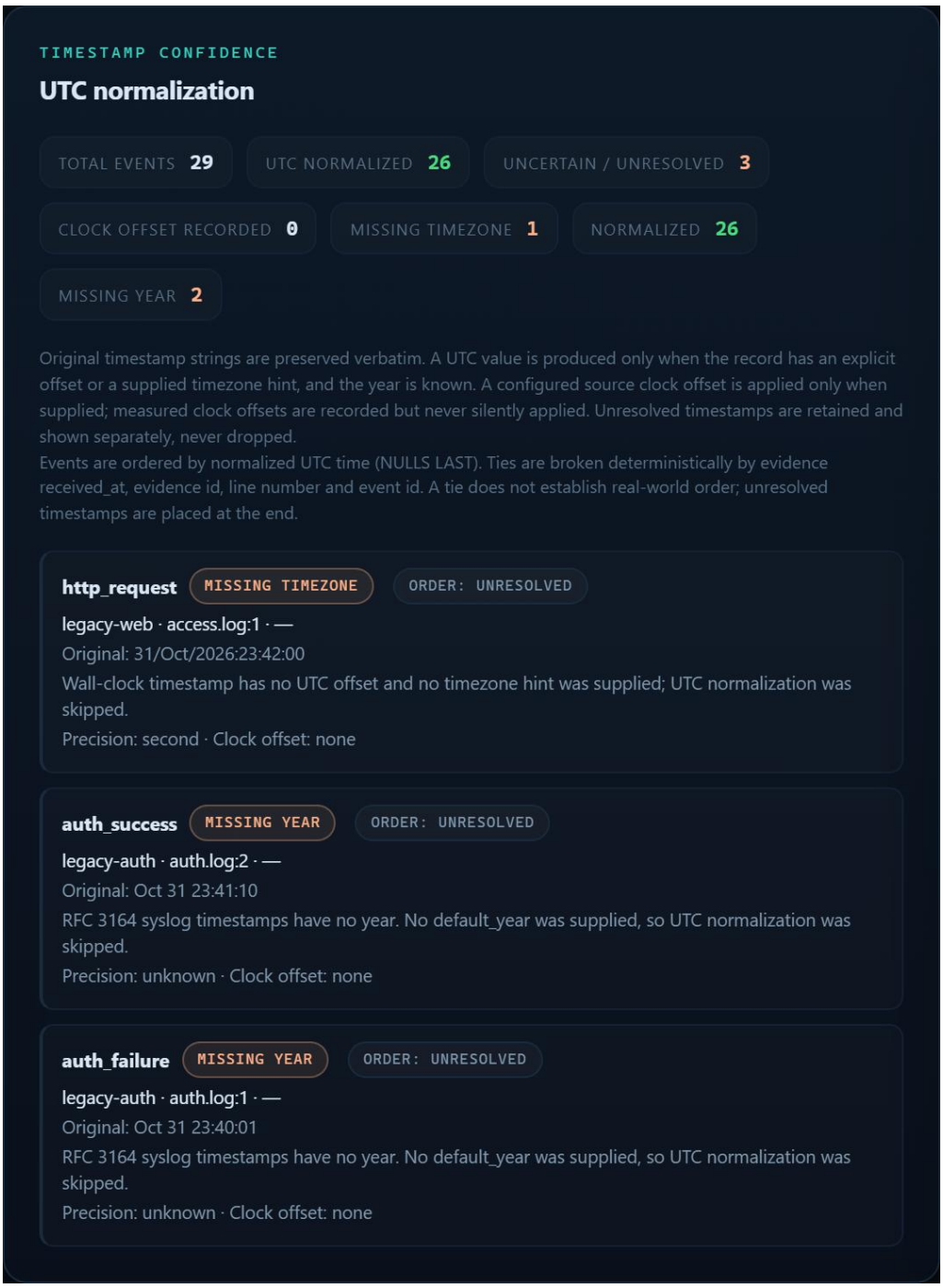


Figure 7.11 UTC normalization

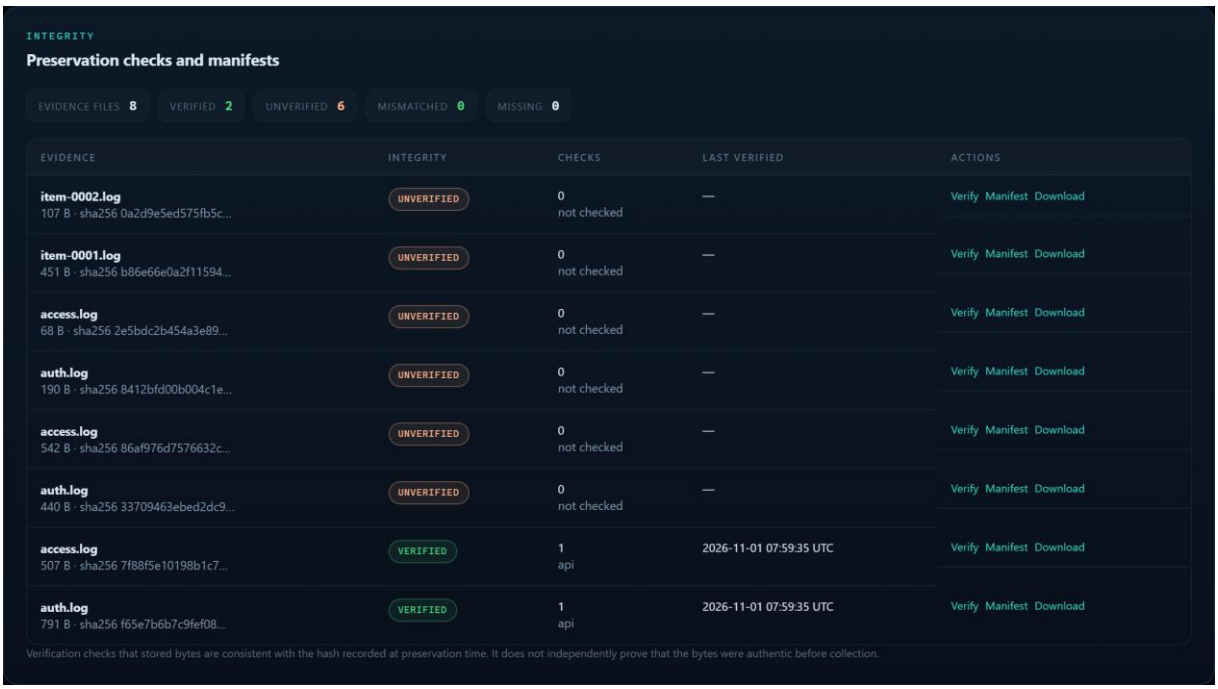


Figure 7.12 Preservation checks

Figure 7.13 is a long capture of the audit trail. It is split across three pages below so that it stays readable; the image itself is unchanged.




Evidence Verified API
2026-11-01 07:59:35 UTC · access.log · actor 127.0.0.1
{
"source": "api",
"outcome": "verified",
"duration_ms": 1,
"actual_sha256": "7f88f5e10198b1c7aa4a9b8f1061567d93be1a16ba8f3dccde865381d2186722",
"expected_sha256": "7f88f5e10198b1c7aa4a9b8f1061567d93be1a16ba8f3dccde865381d2186722"
}
Evidence Verified API
2026-11-01 07:59:35 UTC · auth.log · actor 127.0.0.1
{
"source": "api",
"outcome": "verified",
"duration_ms": 1,
"actual_sha256": "f65e7b6b7c9fef0877c55671266810748f20c4d0ad983118cc25e86c14a3e22e",
"expected_sha256": "f65e7b6b7c9fef0877c55671266810748f20c4d0ad983118cc25e86c14a3e22e"
}
Evidence Parsed API
2026-11-01 07:59:35 UTC · access.log · actor 127.0.0.1
{
"parser_name": "web_access_common_combined",
"parsed_records": 1,
"parser_version": "1.1.0",
"parsing_status": "completed",
"issues_recorded": 1,
"records_rejected": 0
}
Evidence Preserved API
2026-11-01 07:59:35 UTC · access.log · actor 127.0.0.1
{
"size_bytes": 68,
"duplicate_allowed": false,
"stored_relative_path": "evidence/eb84f6b0-95f5-4091-a38a-bd7fbd8239cf/d947b7ae-ebae-41c3-8e68-77c6b42688a7/access.log",
"duplicate_of_evidence_id": null
}
Correlations Generated SYSTEM
2026-11-01 07:59:35 UTC · actor unknown
{
"created": 2,
"updated": 50,
"events_scanned": 24,
"preserved_dismissed": 0,
"time_window_seconds": 1800
}
Evidence Parsed API
2026-11-01 07:59:35 UTC · auth.log · actor 127.0.0.1
{
"parser_name": "linux_auth_syslog",
"parsed_records": 2,
"parser_version": "1.1.0",
"parsing_status": "completed",
"issues_recorded": 2,
"records_rejected": 0
}
Evidence Preserved API
2026-11-01 07:59:35 UTC · auth.log · actor 127.0.0.1
{
"size_bytes": 190,
"duplicate_allowed": false,
"stored_relative_path": "evidence/eb84f6b0-95f5-4091-a38a-bd7fbd8239cf/8adc84b6-2e60-47bf-8738-cd8fa37d37e3/auth.log",
"duplicate_of_evidence_id": null
}
Correlations Generated SYSTEM
2026-11-01 07:59:35 UTC · actor unknown
{
"created": 10,
"updated": 40,
"events_scanned": 23,
"preserved_dismissed": 0,
"time_window_seconds": 1800
}
Evidence Parsed API
2026-11-01 07:59:34 UTC · access.log · actor 127.0.0.1
{
"parser_name": "web_access_common_combined",
"parsed_records": 5,
"parser_version": "1.1.0",
"parsing_status": "completed",
"issues_recorded": 0,
"records_rejected": 0
}
Evidence Preserved API
2026-11-01 07:59:34 UTC · access.log · actor 127.0.0.1
{
"size_bytes": 542,
"duplicate_allowed": false,
"stored_relative_path": "evidence/eb84f6b0-95f5-4091-a38a-bd7fbd8239cf/5edd3597-4d82-4147-b3fb-904bcb68e70c/access.log",
"duplicate_of_evidence_id": null
}



Figure 7.13 Evidence lifecycle audit (shown in three parts)

7.6 Agent Fleet

The fleet page lists registered agents with their health state. The demonstration agent, lab-edge-01.cybertrace.demo, is online with two batches and five events delivered. Its detail view shows the sources it collects, the batches it has sent and any errors it has reported.

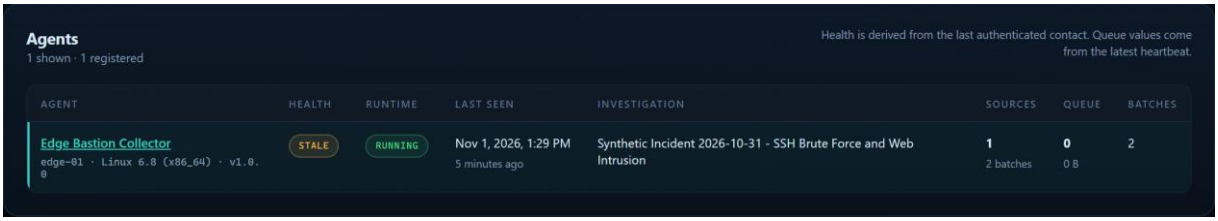


Figure 7.14 Agent list

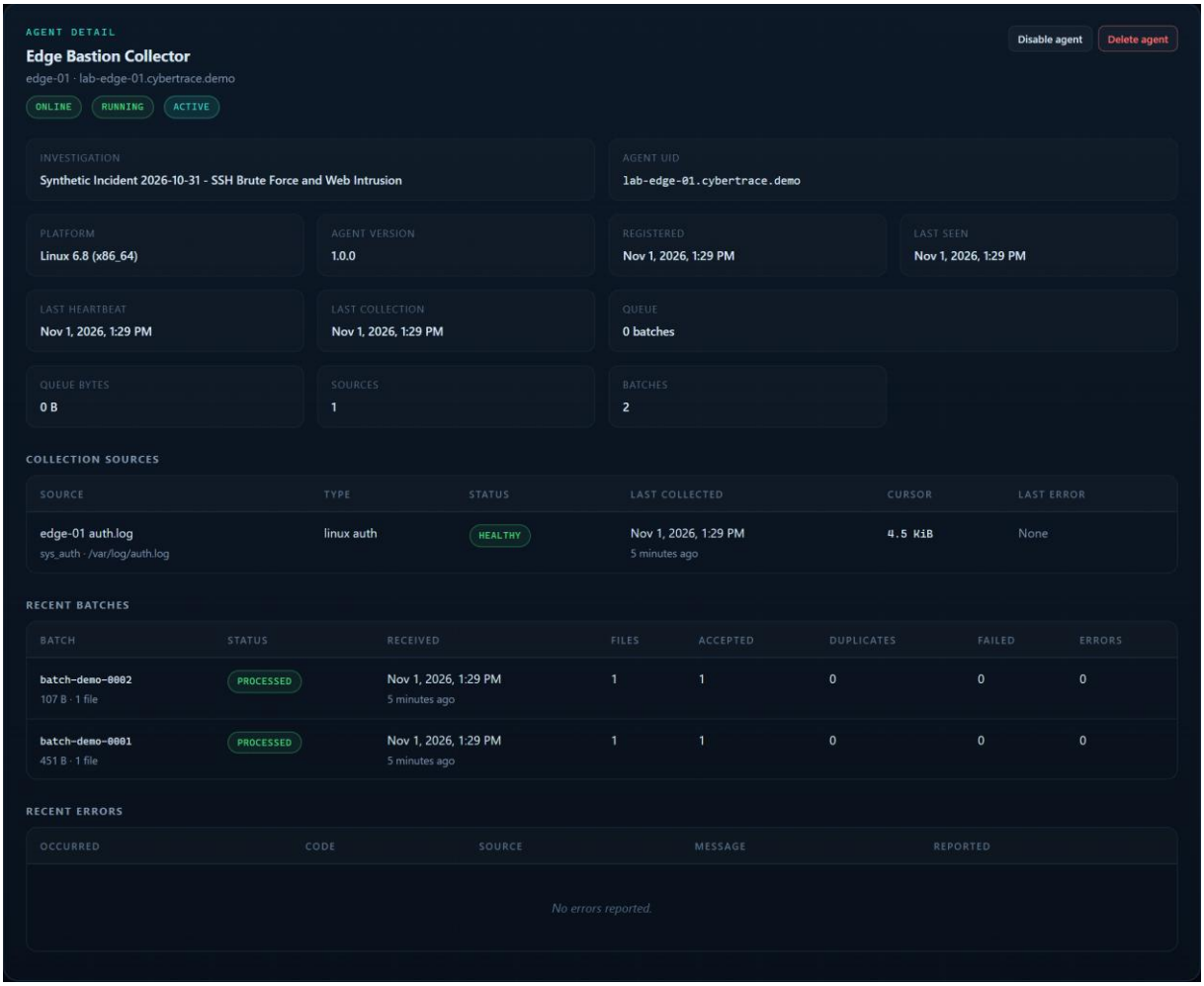


Figure 7.15 Agent detail: batches, sources and errors

7.7 Workspace

The workspace is the original evidence-handling page. It holds the import form, where the investigator sets the source type, hostname, timezone hint, default year and collection time, the list of collected files with their hashes, and a timeline panel.



Figure 7.16 Workspace heading

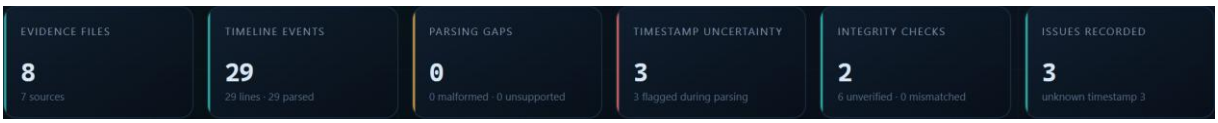


Figure 7.17 Workspace metrics

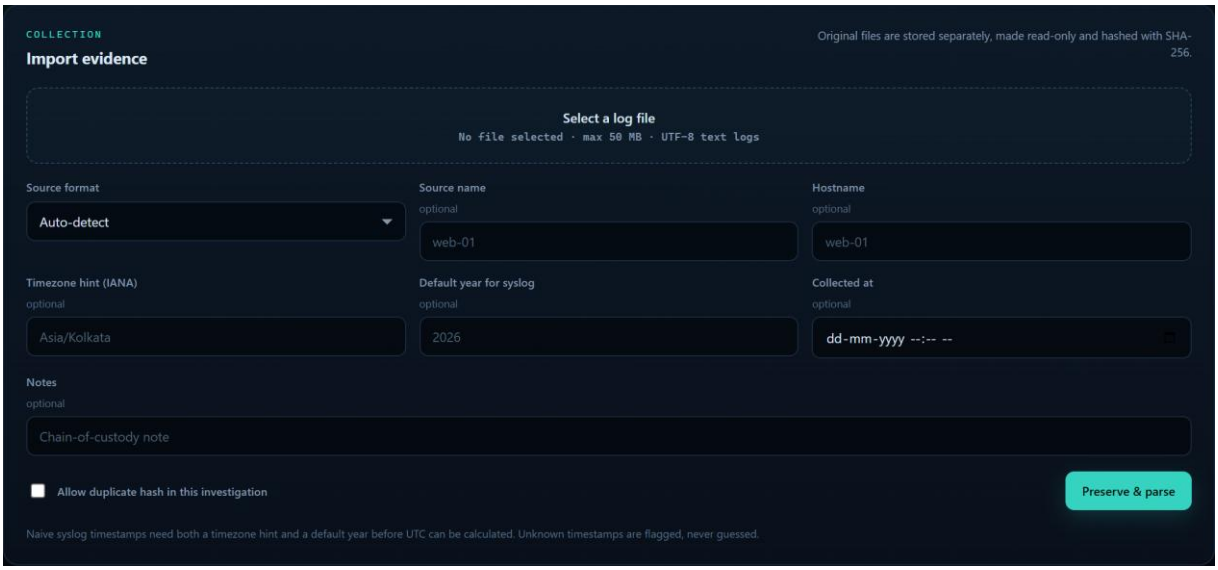


Figure 7.18 Import evidence form

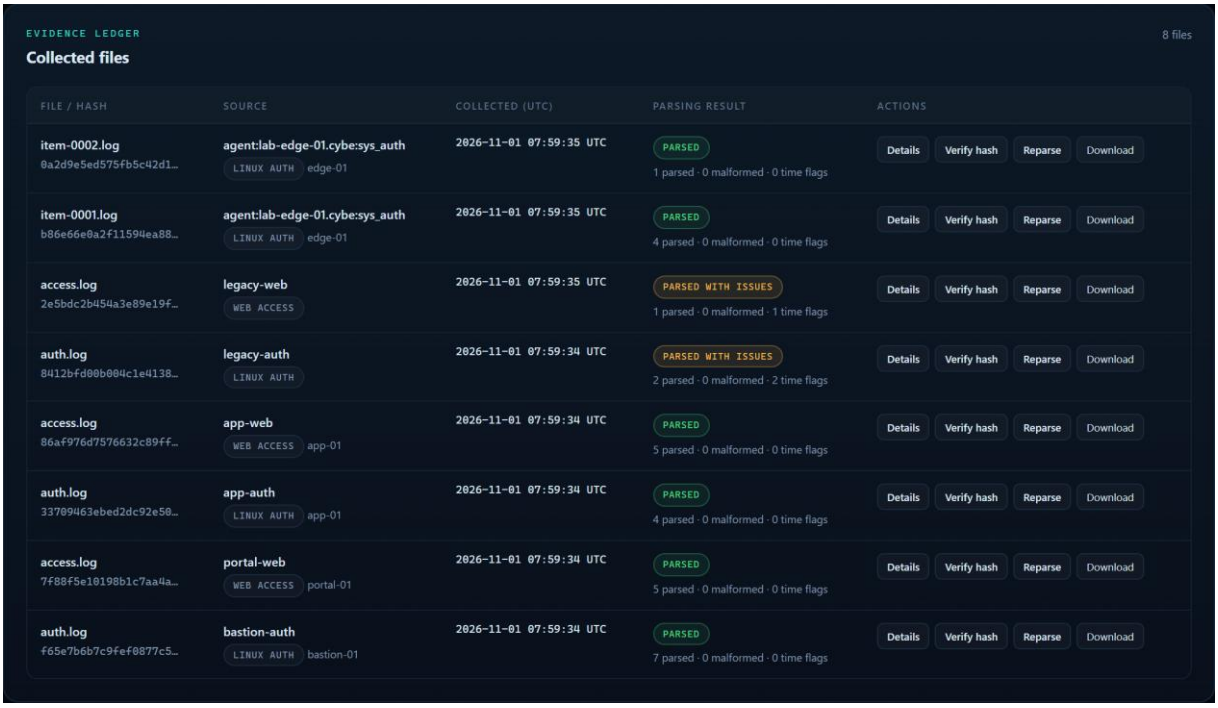


Figure 7.19 Collected files

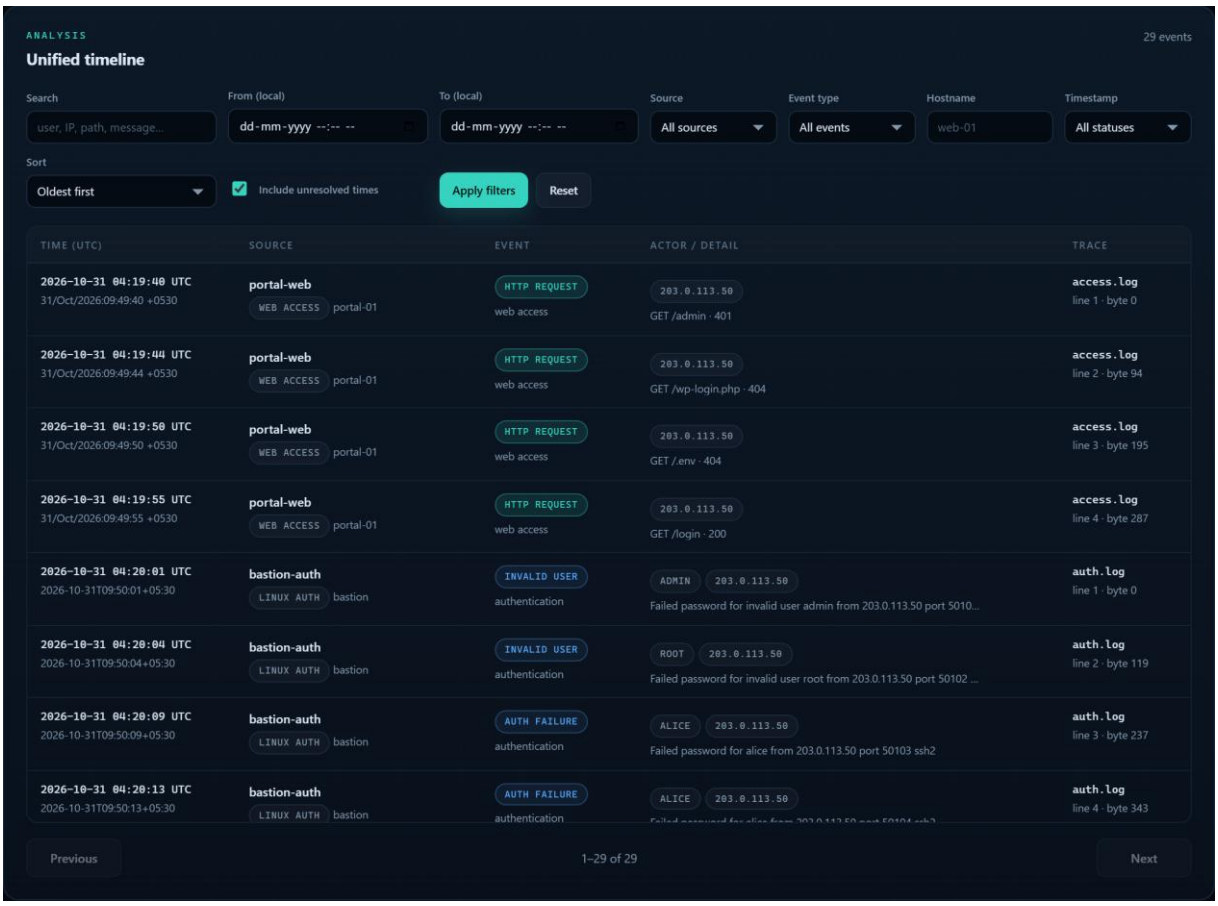


Figure 7.20 Timeline panel

7.8 Discussion

The demonstration dataset exercises the main flows of the system: ingestion of 8 files, 29 normalized events, 52 correlations, timestamp uncertainty reporting, one closed byte-range gap and delivery from an agent. The results agree with the test suites (81 backend tests, 10 agent tests, 31 of 31 end-to-end checks).

Three observations stand out. First, putting the quality indicators beside the timeline changes how the data is read: an event with an unresolved timestamp is visibly different from one with a normalized timestamp, and a gap is visible without going looking for it. Second, keeping the original evidence separate from derived rows made it practical to reparse after a fix, since the hash is verified first and only derived rows are replaced. Third, the explanation attached to each correlation makes it possible to check a link against the log lines instead of trusting it.

The limits are equally clear. Correlations are heuristic and a shared IP address can be coincidental. A hash check confirms consistency with the stored baseline and nothing more. Timestamp resolution depends on the hints supplied. The evaluation uses small synthetic data, and the Docker runtime remains unverified.

Chapter 8. Conclusion, Limitations and Future Work

8.1 Conclusion

CyberTrace provides one auditable workflow for log-based investigations: preservation, parsing, UTC normalization with explicit uncertainty, rule-based correlation, evidence-quality reporting and endpoint collection. Its main value is not that it shows logs together, which a spreadsheet can also do, but that it keeps visible the things that decide how far the evidence can be trusted: integrity, parsing failures, timestamp uncertainty, collection gaps, duplicates and the reasoning behind each correlation.

The system is validated by 81 backend tests, 10 agent tests and a 31-check synthetic end-to-end run, and was demonstrated on a seeded investigation. It should be treated as an academic prototype for controlled investigations and teaching, not as a production forensic platform.

8.2 Limitations

- Correlations are heuristic and can produce false positives. Shared IP addresses can be coincidental, and hostname-only matching is weak.
- Missing or uncertain timestamps limit what can be said about ordering. The system does not infer missing time zones or correct clock skew.
- Coverage describes collected data, not all host activity. Zero collected bytes do not prove that nothing happened.
- Hash verification does not prove authenticity before collection. The audit trail is not signed or tamper-evident at the database level, and manifests are not a chain-of-custody record.
- There is no investigator login, authorization or tenancy. A compromised agent token can submit batches to its assigned investigation.
- Evidence is not encrypted at rest by the application, and no malware scanning is done.
- Only two log families are parsed. Auto-detection can misclassify unusual mixed files.
- No machine learning or behavioral scoring is used.
- Report generation and free-text search are suited to small cases.
- The agent is Linux-oriented, has no auto-update and cannot respond to incidents.
- No production deployment, load test, backup and restore drill, Linux soak test or Docker runtime check has been done.

8.3 Future Work

The following would be natural next steps. They are proposals, not verified features.

- Investigator authentication and role-based access control.
- Signed evidence manifests and tamper-evident audit storage.
- Encryption at rest and key management.
- More log formats (for example Windows Event Log and cloud audit logs) and more operating systems.
- Configurable correlation rules and richer timeline analysis.
- Asynchronous report generation and better performance on large investigations.
- Automated clock-offset analysis with investigator-controlled application.
- Agent auto-update and stronger fleet management.

- Production hardening: monitoring, backup and recovery, a verified Docker runtime and a CI pipeline.
- Evaluation on larger, independently prepared datasets.

8.4 Team

Member	Contribution
Kishore Parasuraman	Technical development, Python-based processing, log analysis, timestamp normalization and evidence integrity verification.
Jabez Gabrilraj	Contributed to the development of CyberTrace.

References

[1] NIST Special Publication 800-86, Guide to Integrating Forensic Techniques into Incident Response. https://csrc.nist.gov/pubs/sp/800/86/final

[2] OWASP Logging Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[3] Wazuh documentation. https://documentation.wazuh.com/

[4] Velociraptor documentation. https://docs.velociraptor.app/

[5] Timesketch. https://timesketch.org/

[6] Plaso (log2timeline) documentation. https://plaso.readthedocs.io/

[7] FastAPI documentation. https://fastapi.tiangolo.com

[8] SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org

[9] Alembic documentation. https://alembic.sqlalchemy.org

[10] PostgreSQL documentation. https://www.postgresql.org/docs

[11] CyberTrace project repository. https://github.com/aiori-hackathon/A3-PS013-TC189.git

Appendix A. API Endpoint Inventory

All routes use the /api prefix. Agent routes marked "bearer" need the per-agent token; routes marked "enrollment" need the X-Enrollment-Token header.

Area	Route	Purpose
Health	GET /health	Database, schema, storage and version status
Investigations	POST /investigations	Create an investigation
	GET /investigations	List investigations with counts
	GET /investigations/{id}	Investigation detail
	GET /investigations/{id}/summary	Evidence, event and issue counters and collection range
	GET /investigations/{id}/sources	Registered evidence sources
Evidence	POST /investigations/{id}/evidence	Upload, hash, preserve, detect and parse
	GET /evidence	List evidence in an investigation
	GET /evidence/{id}	Evidence metadata and parsing summary
	GET /evidence/{id}/status	Event and issue counters
	GET /evidence/{id}/issues	Parsing issues with filters
	GET /evidence/{id}/download	Download the preserved original
	GET /evidence/{id}/manifest	Canonical manifest for one file
	GET /evidence/{id}/verifications	Verification history
	POST /evidence/{id}/verify	Recompute and record the SHA-256
	POST /evidence/{id}/reparse	Verify, then reparse from the original
Timeline	GET /timeline	Search, filter, sort and paginate events
	GET /events/{id}	Full event with raw source line
	GET /events/{id}/correlations	Correlations that include the event
Correlations	POST /investigations/{id}/correlations	Run the correlation rules
	GET /investigations/{id}/correlations	List correlations
	GET /correlations/{id}	Correlation detail with explanation
	POST /correlations/{id}/dismiss	Dismiss with a reason
Agents	POST /agents/register	Enroll or re-enroll (enrollment)
	GET /agents/me	Own identity, health and sources (bearer)
	PUT /agents/sources	Synchronize the source inventory (bearer)
	POST /agents/heartbeat	Runtime status, queue depth and errors (bearer)
	POST /agents/batches	Upload a manifest and files (bearer)
	GET /agents	List agents
	GET /agents/health	Fleet health and queue counters
	GET /agents/{id}	Agent detail

Area	Route	Purpose
	GET /agents/{id}/sources	Source inventory of one agent
	GET /agents/{id}/batches	Batch delivery history
	POST /agents/{id}/disable	Disable an agent (enrollment)
	POST /agents/{id}/enable	Re-enable an agent (enrollment)
	DELETE /agents/{id}	Delete registration and history; evidence stays
Quality	GET /quality/overview	Investigation-level totals
	GET /quality/investigations/{id}/coverage	Per-source coverage for a time window
	GET /quality/investigations/{id}/gaps	Open and closed gaps
	GET /quality/investigations/{id}/parsing	Parser counters and outcomes
	GET /quality/investigations/{id}/timestamps	Timestamp status and uncertain samples
	GET /quality/investigations/{id}/integrity	Integrity state per file
	GET /quality/investigations/{id}/duplicates	Duplicate groups and canonical links
	POST /quality/investigations/{id}/reconcile	Rebuild duplicate links
	GET /quality/investigations/{id}/audit	Audit events with filters
	GET /quality/evidence/{id}/audit	Audit events for one file
	GET /quality/investigations/{id}/manifest	Canonical investigation manifest
Reports	GET /investigations/{id}/report.pdf	PDF investigation report
	GET /investigations/{id}/export/{dataset}.csv	CSV dataset export

Appendix B. Database Tables

Table	Purpose
investigations	Investigation case information
evidence_sources	Sources associated with an investigation
evidence_files	Preserved evidence metadata and hashes
parsed_events	Normalized timeline events
parsing_results	Parsing outcomes and counters
parsing_issues	Malformed, unsupported or flagged records
event_correlations	Correlation groups and explanations
event_correlation_members	Events belonging to a correlation
reconciliation_runs	Duplicate reconciliation activity
agents	Registered collection agents
agent_sources	Log sources configured for each agent
agent_batches	Agent upload batches
agent_batch_items	Evidence segments within a batch
source_collection_windows	Collection-window information used for coverage
collection_gaps	Observed or inferred collection gaps
evidence_audit_events	Evidence lifecycle audit records

Migrations are applied with python -m alembic upgrade head from the backend directory. The current head revision is f42a00000001.

Appendix C. Test Inventory

Backend test modules (81 tests passing):

- test_api_agents.py
- test_api_investigations_health.py
- test_correlation.py
- test_evidence_upload.py
- test_parsers.py
- test_quality.py
- test_reports.py
- test_samples.py
- test_time_handling.py
- test_timeline.py
- conftest.py (shared fixtures)

Agent tests: 10 passing, in agent/tests/test_agent.py. End-to-end: scripts/e2e_synthetic_check.py, 31 of 31 checks passing.

Appendix D. Deployment Notes

D.1 Server

The server is deployed with docker-compose.yml (a database service and an API service under the project name cybertrace), backend/Dockerfile (non-root runtime), backend/docker-entrypoint.sh (wait for the database, run migrations, start the server) and .env.example (placeholder configuration). The real .env should not be committed or shared. The agent enrollment token is generated with:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

and set as AGENT_ENROLLMENT_TOKEN in the server environment. Registration stays disabled while it is unset. The API should be served over HTTPS, for example behind nginx or Caddy.

Docker Engine was not available when the project was verified, so the image build and container runtime were not tested. The verification that was carried out is recorded in reports/DOCKER_SETUP_AND_VERIFICATION.md.

D.2 Agent

1. Create a system account (cybertrace) with no login shell and a state directory such as /var/lib/cybertrace-agent.
2. Create a virtual environment under /opt/cybertrace-agent and install the agent package into it.
3. Copy config.example.json to /etc/cybertrace-agent/config.json and set server_url, investigation_id, state_dir, display_name, sources and start_position.
4. Write the enrollment token to a separate file readable only by the agent account.
5. Give the account read access to the log files (group membership or ACLs).
6. Install the supplied systemd unit and start the service.

Appendix E. Agent Configuration Reference

The configuration is strict JSON. Unknown keys are rejected so that a typing mistake cannot silently disable collection.

Key	Default	Meaning
server_url	required	HTTPS origin of the server, without /api
api_prefix	/api	API prefix on the server
enrollment_token_file	none	Path to the enrollment token; needed only for first enrollment or re-enrollment
state_dir	required	Absolute directory for SQLite state, spool, quarantine, lock and agent token
investigation_id	required	Investigation UUID; must match the server
agent_uid	derived	Stable identifier; derived from /etc/machine-id if omitted
display_name	hostname	Label shown in the dashboard
heartbeat_interval_seconds	60	Heartbeat cadence (5 to 86400)
collection_interval_seconds	30	Collection cadence (1 to 86400)
flush_interval_seconds	30	Upload cadence (1 to 86400)
max_backoff_seconds	300	Maximum retry delay
http_timeout_seconds	30	Per-request timeout
max_batch_bytes	1048576	Maximum bytes per upload batch
max_batch_files	20	Maximum segments per batch
max_segment_bytes	262144	Maximum bytes per spooled segment
max_buffer_bytes	268435456	Maximum spooled bytes waiting for upload
start_position	end	beginning or end for a source with no cursor
allow_insecure_localhost	false	Permit HTTP only for loopback addresses
tls.ca_file	system trust	Optional CA bundle
tls.client_cert_file, tls.client_key_file	none	Optional mutual-TLS client certificate pair